

Diseño y simulación de un robot manipulador serial de seis grados de libertad para automatización logística.

Ignacio Martín Duci
ignacio8martin@gmail.com

Proyecto Final de Estudios (00441),
Facultad de Ingeniería,
Universidad Nacional de Cuyo,

Mendoza, Argentina

Director: Ing. Eric Sánchez

Septiembre 2026

Resumen

Se presenta el diseño y la simulación de un robot manipulador serial de seis grados de libertad orientado a la automatización de procesos logísticos, específicamente a la transferencia de objetos entre contenedores (*bin-to-bin transfer*). La motivación surge del crecimiento sostenido del mercado de robótica para depósitos, con tasas de expansión proyectadas entre el 17 % y el 22 % anual para Latinoamérica, región que presenta las mayores oportunidades relativas de automatización.

El desarrollo cubre el ciclo completo desde el modelado hasta la integración en software de robótica: se formula la cinemática directa mediante la convención Denavit-Hartenberg, se obtiene una solución analítica cerrada de la cinemática inversa con hasta 512 configuraciones articulares válidas, y se caracteriza el espacio de trabajo y las singularidades geométricas del manipulador (hombro, codo y muñeca). Sobre esta base se implementa un módulo de generación de trayectorias con interpolación en espacio articular y cartesiano, y escalado automático de velocidades y aceleraciones para respetar los límites cinemáticos. Los modelos CAD de cada eslabón se diseñaron en *Fusion 360* y se integraron como mallas STL al entorno de simulación en MATLAB con la *Robotics Toolbox* de Peter Corke. Finalmente, la cinemática y la generación de trayectorias se portaron a *Robot Operating System 2* (ROS 2), construyendo una arquitectura de nodos con interfaces personalizadas, modelo URDF, visualización en RViz 2 e interfaz gráfica de usuario desarrollada en PyQt5.

La simulación de la tarea completa de *bin-to-bin transfer* confirma el cumplimiento de los límites cinemáticos impuestos y la coherencia entre los modelos MATLAB y ROS 2.

Palabras clave: robot manipulador serial, seis grados de libertad, Denavit-Hartenberg, cinemática inversa analítica, generación de trayectorias, Jacobiano geométrico, singularidades, MATLAB, Robotics Toolbox, Peter Corke, Fusion 360, ROS 2, URDF, RViz 2, PyQt5, automatización logística, simulación robótica.

Agradecimientos

A la Facultad de Ingeniería de la Universidad Nacional de Cuyo, por los espacios de estudio y formación brindados. A sus docentes, por transmitir no solo conocimientos técnicos, sino también la pasión por la ingeniería.

A mi director, el Ing. Eric Sánchez, por acompañarme a lo largo de todo este proyecto con dedicación y generosidad. Por guiarme cuando el camino no era claro, por motivarme en los momentos difíciles, por confiar en el proyecto incluso antes de que tomara forma, y por el tiempo que destinó con genuino interés.

A mi familia y amigos, por estar presentes en cada etapa de esta carrera y en mi vida. Por el sostén incondicional, por celebrar cada logro como propio y por las horas compartidas dentro y fuera de la facultad. Su compañía fue, y es, indispensable.

A Dios.

Índice

1. Introducción	6
1.1. Motivación y mercado	6
1.2. Propuesta	6
2. Objetivos y beneficiarios	9
3. Descripción de la tarea y especificaciones del robot	10
4. Diseño conceptual preliminar	13
5. Modelado cinemático	15
5.1. Modelado cinemático	15
5.2. Cinemática inversa	17
5.3. Convenciones de representación cartesiana	24
5.4. Análisis y verificación de espacio de trabajo	24
5.5. Análisis de singularidades	29
6. Generación de trayectorias	30
6.1. Generación de trayectorias	30
6.2. Escalamiento de velocidades y aceleraciones	30
7. Diseño y adaptación de CAD	32
7.1. Visualización del modelo en MATLAB	32
7.2. Diseño de piezas en <i>Fusion 360</i>	32
7.2.1. Base	33
7.2.2. Eslabón 1	33
7.2.3. Eslabón 2	34
7.2.4. Eslabón 3	34
7.2.5. Eslabón 4	35
7.2.6. Eslabón 5	35
7.2.7. Eslabón 6 —Herramienta	36
7.3. Diseño alternativo: apertura unidireccional de eslabones	36
8. Simulación y verificación de tarea	38
9. Integración en entorno ROS 2	42
9.1. Arquitectura del sistema	42
9.2. Interfaces personalizadas	43
9.3. Interrelación entre nodos	45
9.4. Modelo URDF	46
9.5. Cinemática	46
9.6. Generación de trayectorias	47
9.7. Teaching	47
9.8. Interfaz gráfica	48
10. Verificación de productividad en entorno ROS 2	53
10.1. Etapa 1: <i>Teaching</i> de la trayectoria	53
10.2. Etapa 2: Recorrido de la trayectoria	54
10.3. Etapa 3: Análisis cinemático	54
11. Conclusiones y trabajo futuro	57

12. Uso de agentes de inteligencia artificial	58
12.1. Metodología de uso de inteligencia artificial	58
12.2. MATLAB	58
12.3. ROS 2	59
12.4. Desarrollo del informe	59
12.5. Uso general	59
12.6. Formato de implementación	59
A. Verificación y Validación de los Módulos de Software de MATLAB y ROS 2	60
A.1. Cinemática Inversa	60
A.2. Singularidades	61
A.3. Generación de trayectorias	62
A.4. Integración en ROS 2	63

1. Introducción

1.1. Motivación y mercado

La automatización e informatización de procesos de servicios es una tendencia que, con el pasar de los años, incrementa su presencia en los diferentes mercados, brindando mejoras en eficiencia, precisión, seguridad, operatividad y trazabilidad. Al mismo tiempo, la adopción de estas tecnologías resta competitividad a aquellas entidades que, por disponibilidad, costo o falta de personal capacitado, no pueden implementarlas.

A partir de datos de la consultora *Grand View Research* sobre las proyecciones de mercado 2023–2030 para la robótica aplicada a depósitos logísticos (*warehouses*) a nivel mundial [1], en Estados Unidos [2], China [3], Latinoamérica [4] y Brasil [5], pueden elaborarse las siguientes conclusiones:

1. Todos los mercados proyectan un crecimiento anual compuesto de entre 17 % y 22.1 %, lo cual se considera por encima de un escenario favorable.
2. Comparativamente, Latinoamérica proyecta un escenario de crecimiento mayor al de Estados Unidos, China y al mundo en su totalidad.
3. Al analizar un país de la región con datos disponibles, como Brasil, puede verse que proyecta un crecimiento similar al de China en el período analizado.

Bajo estas conclusiones, puede entenderse que el mercado continuará creciendo, y que países con menores niveles de automatización e informatización serán los que presentarán mejores oportunidades para este campo. Si se considera la situación actual de Argentina, orientada a un modelo productivo con menor carga impositiva, Argentina puede posicionarse como plataforma estratégica para fabricar y exportar robótica de servicio.

En el contexto de la automatización de los procesos logísticos, la implementación de robots manipuladores es esencial para lograr movimientos en el espacio con alto grado de flexibilidad que permitan la reorganización de paquetes con alta precisión y eficiencia.

El desarrollo desde cero de un robot tipo serie de seis grados de libertad presenta una excelente oportunidad para integrar los conocimientos adquiridos en la carrera. También se presentarán desafíos que requerirán expandir conocimientos y aplicar nuevos conceptos y técnicas con criterio. La robótica serie es un campo profundamente estudiado, por lo que podrá accederse a una gran cantidad de información ya existente.

A lo largo del desarrollo se extrajeron conceptos teóricos de las siguientes referencias: *Introduction to Robotics* (Craig, 2021), *Robotics, Vision and Control* (Corke, 2023), *Artificial Intelligence: A Modern Approach* (Russell & Norvig, 2021), *Learning ROS 2* (Martínez & Fernández, 2024) y *Learning Robotics Simulation Using ROS and Gazebo* (Hyman, 2018).

1.2. Propuesta

En el presente trabajo se desarrolla el diseño y la simulación de un robot manipulador serial de seis grados de libertad orientado a una aplicación específica: la transferencia de objetos entre contenedores (*bin-to-bin transfer*).

Esta aplicación se plantea como un caso representativo dentro de procesos de manipulación en depósitos logísticos automatizados (*micro-fulfillment warehouses*), permitiendo validar el desempeño del sistema en tareas típicas de pick-and-place.

La selección del caso de estudio se inspira en desarrollos industriales existentes, como el presentado en [YouTube](#). No obstante, la solución propuesta en este trabajo es de desarrollo propio, enfocada en el modelado cinemático, la generación de trayectorias y la simulación del comportamiento del manipulador.

En la figura 1 puede verse un esquema de la aplicación descrita.

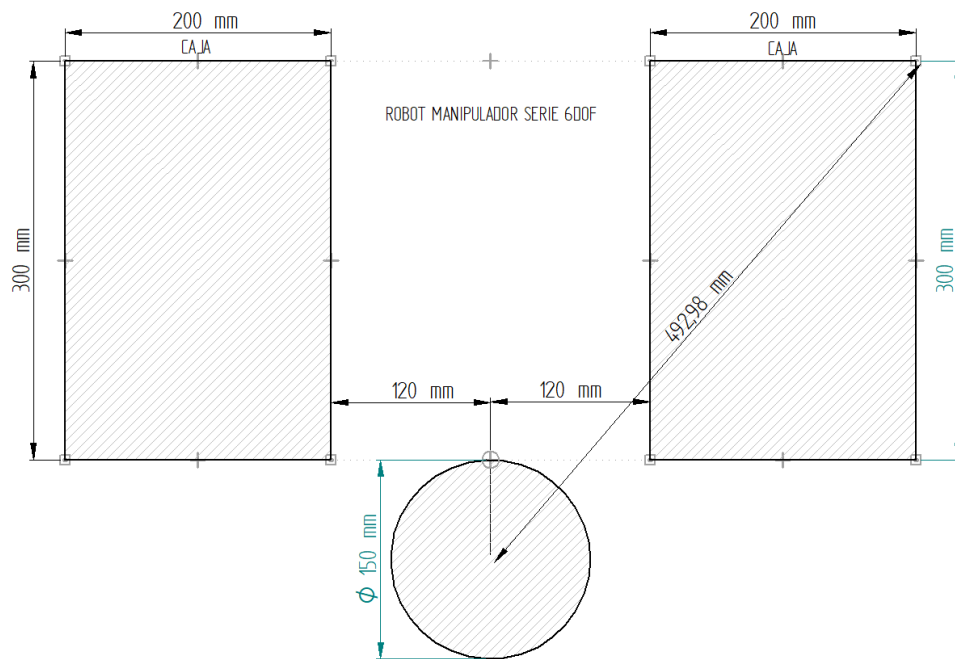


Figura 1: Esquema de la aplicación - vista superior.

Mínimamente, se propone que el robot tenga un alcance de 500 mm, similar a las características del robot comercial [UR3e](#). En la figura 2 se puede ver un esquema general de la morfología del robot.

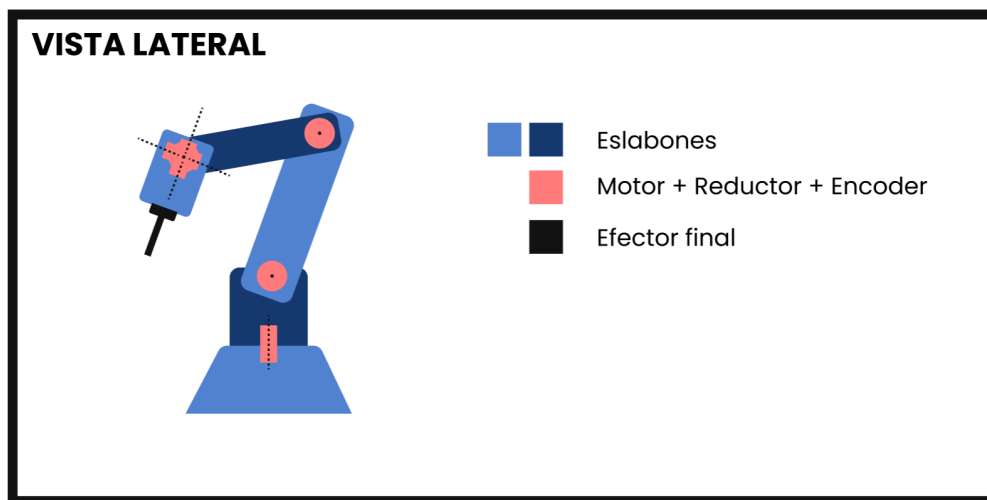


Figura 2: Esquema del robot tipo serie de 6 GDL propuesto.

La tarea del robot consiste cronológicamente en lo siguiente:

1. Roto-traslación a la postura de espera de forma rápida.
2. Roto-traslación a la postura sobre la caja de origen a alta velocidad.
3. Roto-traslación para *picking* a baja velocidad.
4. Roto-traslación hacia la postura sobre la caja de origen a baja velocidad.
5. Roto-traslación hacia la postura sobre la caja de destino a alta velocidad.
6. Roto-traslación para *placing* a baja velocidad.

7. Roto-traslación hacia la postura sobre la caja de destino a baja velocidad.
8. Repetir.

2. Objetivos y beneficiarios

Objetivo mínimo: Diseñar y simular un robot manipulador de seis ejes tipo serie con 500 mm de alcance destinado a tareas de *bin-to-bin transfer*, capaz de mover 300 elementos por hora.

Los siguientes objetivos se desarrollan en dos etapas: los ítems 1 a 6 corresponden a la implementación y validación en entorno MATLAB, mientras que los ítems subsiguientes se desarrollan en el entorno ROS 2.

1. Definir las especificaciones del robot y su tarea.
2. Realizar un diseño conceptual preliminar.
3. Resolver analíticamente y validar la cinemática inversa.
4. Analizar y validar el espacio de trabajo.
5. Implementar y validar un programa de generación de trayectorias que incorpore distintas funcionalidades, permitiendo la selección de modos de interpolación (cartesiana y articular).
6. Implementar, simular y validar la ejecución de la tarea definida, asegurando su correcto desempeño y cumplimiento de los objetivos planteados.
7. Desarrollar el modelo CAD del robot y su representación en formato URDF a partir de la parametrización cinemática DH, asegurando la coherencia geométrica entre eslabones, articulaciones y marcos de referencia, e integrar las geometrías como visualización en RViz 2.
8. Integrar en ROS 2 la cinemática directa e inversa, la generación de trayectorias, el sistema de *teaching*, la gestión de trayectorias y una interfaz gráfica de usuario que permita el movimiento manual, la enseñanza de puntos y el análisis de variables cinemáticas.

Posibles beneficiarios:

1. **Bodegas y agro-industria mendocina** que requieren automatizar el empaque y la manipulación de insumos sin invertir en robots industriales de alto costo.
2. **Centros de *micro-fulfillment* y PYMES logísticas** donde el brazo puede aumentar el *pick-rate* y reducir tiempos de entrega.
3. **Institutos y universidades locales** (UNCuyo, ITU, UTN) que obtendrán una plataforma didáctica *open-source* para prácticas de mecatrónica y control.

3. Descripción de la tarea y especificaciones del robot

La tarea del manipulador consiste en la transferencia de objetos entre dos contenedores de dimensiones y posiciones predefinidas. Esto puede verse en las figuras 3 y 4, donde también quedan definidos puntos clave de la trayectoria que deberá seguir el robot conforme a lo definido en la sección 1.2.

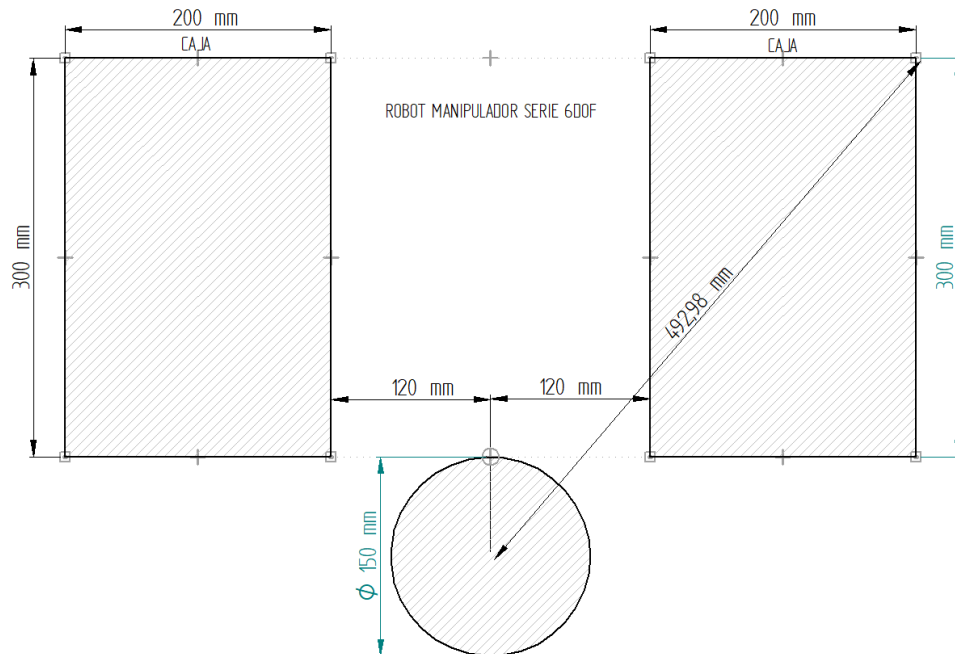


Figura 3: Vista superior del robot y la tarea

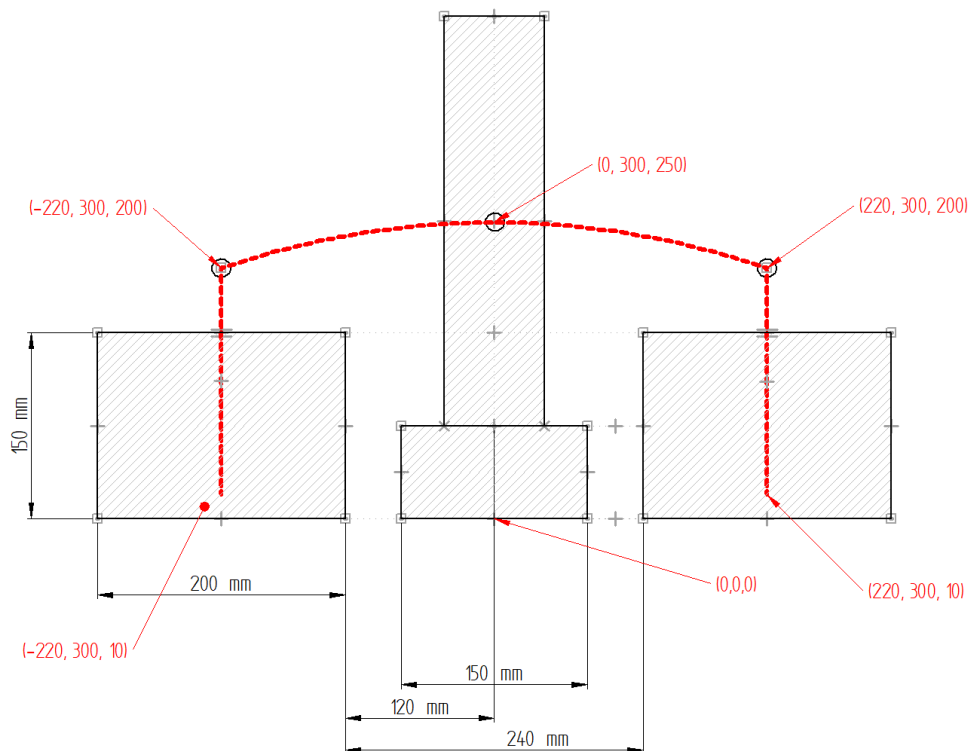


Figura 4: Vista lateral del robot y la tarea

Los objetos serán tomados mediante un sistema de sujeción por vacío (ventosa - *vacuum gripper*) y depositados mediante la liberación del mismo; esto permite relajar los requerimientos de precisión y repetibilidad del manipulador.

Descripción y alcance de la trayectoria

El sistema de transferencia completo requiere la integración de tres componentes: un generador de trayectorias, un sistema de visión artificial y un planificador de alto nivel. En el presente trabajo el alcance se limita al desarrollo del generador de trayectorias para el cálculo de la trayectoria nominal y a la simulación de las trayectorias reactivas, con el fin de verificar la viabilidad de la implementación de la tarea. El sistema de visión artificial y el planificador de trayectoria quedan reservados para desarrollos posteriores.

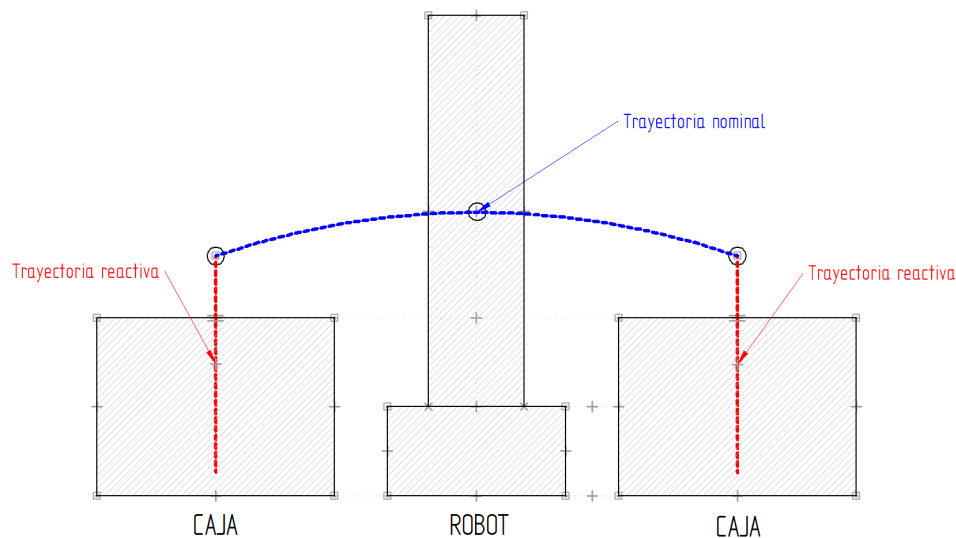


Figura 5: Vista lateral del robot con segmentos de trayectoria nominal y reactiva.

La trayectoria completa se estructura en tres segmentos diferenciados, ilustrados en la figura 5. El primero, de naturaleza **nominal**, conecta los puntos definidos por encima de la caja de origen y la caja de destino. Este segmento es predefinido y reproducible ciclo a ciclo, por lo que puede diseñarse para maximizar la productividad. Los otros dos, de naturaleza **reactiva**, conectan respectivamente el punto sobre la caja de origen con el punto de *picking* y el punto sobre la caja de destino con el punto de *placement*. Estas trayectorias no son fijas, ya que deben adaptarse en cada ciclo a las condiciones del entorno.

Las trayectorias reactivas presentan múltiples restricciones cinemáticas y contextuales: los bordes de la caja, la presencia de otros objetos en su interior, la orientación del objeto a manipular y el espacio disponible para su colocación. Su generación requiere un sistema de reconocimiento de objetos mediante visión artificial que identifique el objeto más adecuado y más cercano al punto de partida, y determine el espacio libre en la caja de destino siguiendo una estrategia de relleno desde el centro hacia el exterior. Como nivel de coordinación superior, un planificador de trayectoria debería gestionar la activación de cada modo según el estado del sistema.

Especificaciones del proyecto

Quedan definidas las especificaciones del manipulador que son objeto del presente trabajo:

- Grados de libertad (GDL - *DOF*): 6.
- Alcance: 500.00 mm
- Rango articulaciones 1 a 5: $\pm 2\pi$
- Rango articulación final: *libre*

Especificaciones para posible extensión del proyecto

Las siguientes especificaciones quedan definidas como referencia para desarrollos posteriores que aborden el diseño mecánico, el control de fuerza y la integración con el entorno físico:

- Capacidad de carga útil: 3.00 kg
- Precisión: 0.5 mm
- Repetibilidad: 0.5 mm
- Temperatura ambiente entre 5 y 35 °C.
- Humedad relativa $\leq 70\%$.
- Bajo nivel de polvo ambiental.
- Iluminación 100-150 lx.
- Entorno sin vibraciones.
- Protección IP54

4. Diseño conceptual preliminar

Con el objetivo de tener un punto de partida en cuanto a la morfología y dimensiones del robot, tomando como referencia al robot [UR3e](#), se define la siguiente estructura:

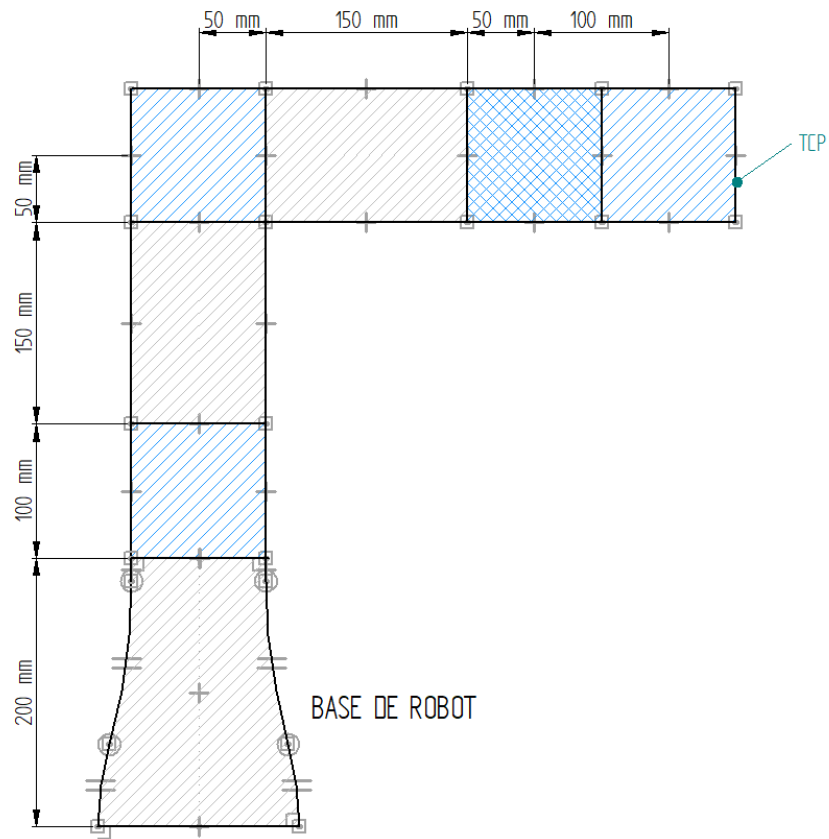


Figura 6: Vista lateral 1 de un esquema del robot

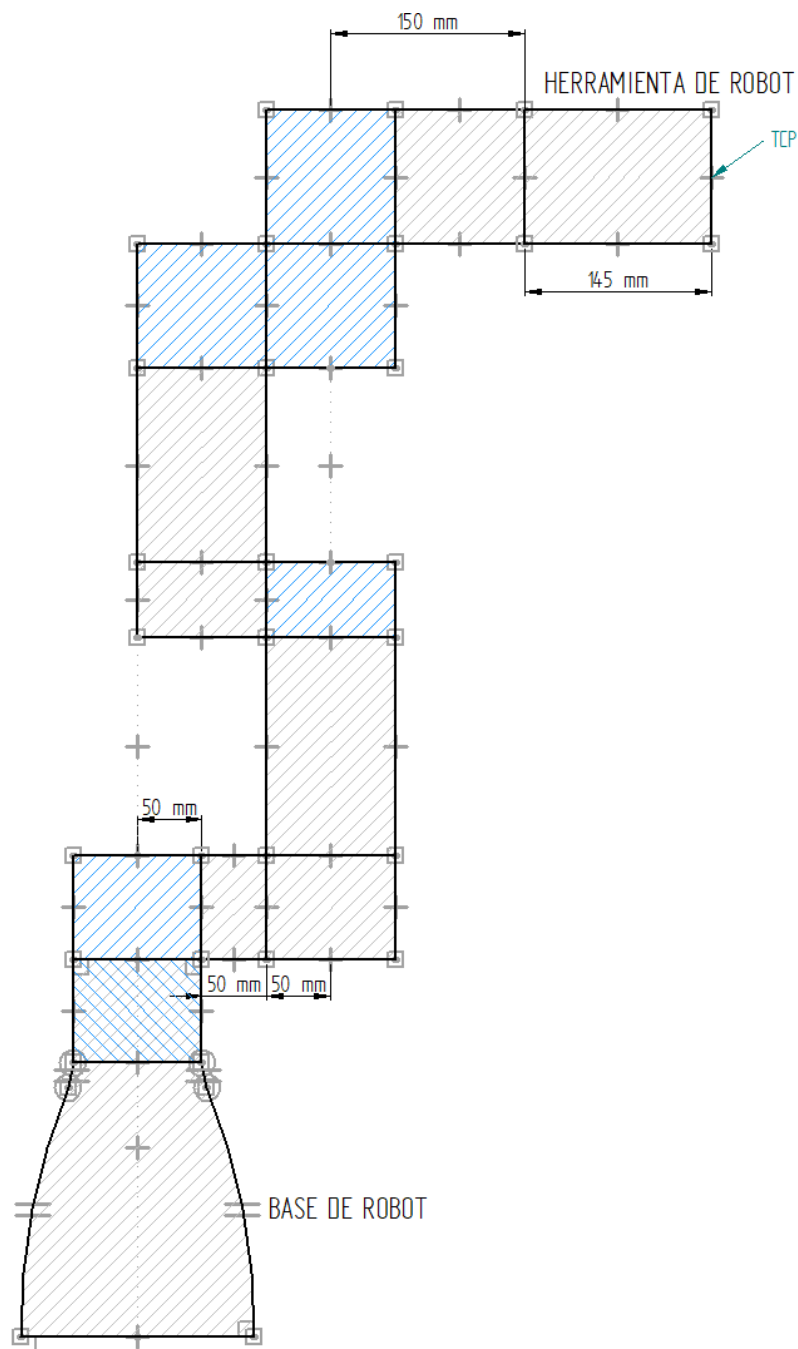


Figura 7: Vista lateral 2 de un esquema del robot

A partir de estas imágenes se obtiene la información necesaria para la elaboración del modelo cinemático. Se ha incorporado una base para permitir que el robot opere de forma segura por encima de las cajas y una herramienta final tipo ventosa (*vacuum-gripper*); para esta última se toma como referencia el modelo *EPick with one suction cup 145x83 mm* del fabricante [Robotiq](#).

5. Modelado cinemático

En esta sección se aborda el modelado cinemático del manipulador, así como la resolución de la cinemática inversa.

El modelado cinemático permite establecer la relación entre las variables articulares del robot y la posición y orientación de su efector final. A partir de este modelo, se plantea la resolución de la cinemática inversa, con el objetivo de determinar las configuraciones articulares requeridas para alcanzar posiciones específicas dentro del espacio de trabajo.

Finalmente, se desarrolla la generación de trayectorias, mediante la cual se define la evolución temporal del movimiento del manipulador entre distintas configuraciones, garantizando continuidad, suavidad y seguridad en su ejecución.

5.1. Modelado cinemático

Para obtener el modelo cinemático se aplica el algoritmo Denavit-Hartenberg estándar, el cual consiste en lo siguiente:

1. Enumerar los $n+1$ eslabones desde 0 hasta n , comenzando desde la base (eslabón fijo) y terminando en el efector final.
2. Identificar los ejes de cada articulación. Si es rotacional, será el eje de giro; si es prismática, será el eje a lo largo del cual se produce el desplazamiento.
3. Enumerar los ejes de 1 a n , comenzando desde el que une el eslabón base con el eslabón 1.
4. Para i de 0 a $n-1$, situar el eje Z_i en el eje de la articulación $i+1$.
5. El eje Z_n se colocará en el extremo del último eslabón en la misma dirección que el eje Z_{n-1} .
6. Situar el origen del sistema de la base $\{S_0\}$ en cualquier punto del eje Z_0 .
7. Para $i = 1$ a n , ubicar el sistema $\{S_i\}$ en la intersección entre el eje Z_i y la recta perpendicular común a los ejes Z_i y Z_{i-1} . En caso de que los ejes Z_i y Z_{i-1} se intersecten, el sistema $\{S_i\}$ se posiciona en dicho punto de intersección.
8. Para $i = 1$ a n , definir el eje X_i a partir del punto donde se estableció el sistema $\{S_i\}$, sobre la recta perpendicular común a los ejes Z_i y Z_{i-1} . Si los ejes Z_i y Z_{i-1} se intersectan, el eje X_i debe ser perpendicular a ambos. Su orientación puede elegirse de forma arbitraria.
9. El eje X_0 puede definirse libremente, siendo conveniente alinearlos con el eje X_1 .
10. Para $i = 0$ a n , definir el eje Y_i de manera que, junto con los ejes X_i y Z_i , forme un sistema de referencia dextrógiro.

Luego se determinan los parámetros que permitirán elaborar una matriz que define al robot en cuestión de la siguiente forma:

Para $i = 1$ a n :

1. θ_i : Ángulo alrededor del eje Z_{i-1} , desde el eje X_{i-1} hasta el eje X_i .
2. d_i : Distancia a lo largo del eje Z_{i-1} , desde el origen del sistema $i - 1$ hasta el eje X_i .
3. a_i : Distancia a lo largo del eje X_i , desde el eje Z_{i-1} hasta el eje Z_i .
4. α_i : Ángulo alrededor del eje X_i , desde el eje Z_{i-1} hasta el eje Z_i .

Donde:

- a_i : Longitud del eslabón.
- α_i : Ángulo de torsión del eslabón.
- d_i : Longitud articular.

- θ_i : Ángulo articular.

Utilizando esta información, los parámetros de Denavit-Hartenberg pueden organizarse en la siguiente forma general:

i	θ_i	d_i	a_i	α_i
1	θ_1	d_1	a_1	α_1
2	θ_2	d_2	a_2	α_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	θ_n	d_n	a_n	α_n

(1)

Finalmente, se ha conseguido describir la geometría del robot y la interrelación entre sus eslabones mediante un algoritmo. El beneficio de esta representación radica en que permite determinar la posición y orientación de cada sistema de referencia a partir del anterior y, por consiguiente, obtener la posición y orientación del efector final en función de las variables articulares. A continuación, se presenta de forma general la matriz de transformación homogénea que relaciona cada sistema de referencia con el anterior.

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Puede entonces determinarse, para un robot de 6 grados de libertad, la posición y orientación del efector final en función de las variables articulares mediante la transformación sucesiva del sistema base hasta el sistema herramienta.

$${}^{base}T_{tcp} = {}^bT_0 {}^0T_1 \dots {}^5T_6 {}^6T_{tcp} \quad (3)$$

De esta forma quedará definido **el modelo cinemático directo**.

Se obtiene información sobre las dimensiones del robot de las figuras 6 y 7 y se elabora el siguiente esquema simplificado con información solo esencial al aplicar el algoritmo DH:

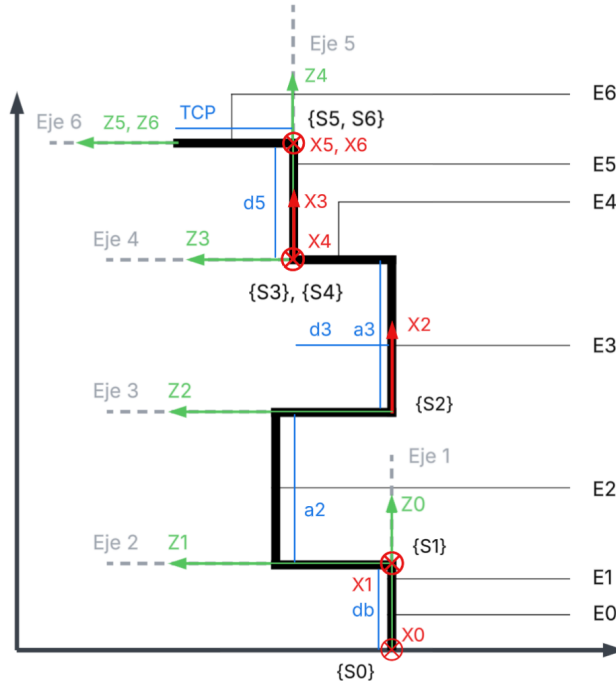


Figura 8: Algoritmo DH aplicado a un esquema simplificado del robot

Teniendo en cuenta que las articulaciones son de tipo rotacional, la variable articular será θ para todos los casos. Se define la matriz Denavit-Hartenberg a partir de la información de la figura 8 y del procedimiento definido en la explicación del algoritmo:

i	$\theta_i[rad]$	$d_i[m]$	$a_i[m]$	$\alpha_i[rad]$
1	q_1	0,25	0	$\frac{-\pi}{2}$
2	$q_2 - \frac{\pi}{2}$	0	0,25	0
3	q_3	0,295	0,25	0
4	$q_4 + \frac{\pi}{2}$	0	0	$\frac{\pi}{2}$
5	q_5	0,1	0	$\frac{-\pi}{2}$
6	q_6	0	0	0

(4)

Notas:

- Mediante la letra " q " se simboliza el parámetro variable.
- La suma o resta a valores de q , como lo es en los casos 2 y 4, da un offset de posición que modifica el punto cero del robot.

5.2. Cinemática inversa

En el desarrollo cinemático de todo robot es fundamental elaborar el modelo de cinemática inversa, el cual permite determinar las variables articulares necesarias para alcanzar una posición y orientación deseadas del efector final. En este trabajo, la resolución se aborda mediante un enfoque analítico, obteniendo expresiones en forma cerrada que relacionan directamente las variables del espacio cartesiano con las variables articulares, evitando el uso de métodos numéricos iterativos.

Se parte de la observación de que el manipulador no cuenta con una muñeca esférica; es decir, los tres ejes correspondientes a la muñeca no se intersectan en un mismo punto, lo que impide hacer uso del método de Pieper.

En esta resolución, se implementan métodos algebraicos para separar incógnitas y determinarlas en función de parámetros conocidos y variables ya determinadas.

Dado que se ha definido el modelo cinemático directo al haber elaborado la matriz Denavit-Hartenberg, se conocen todas las matrices de transformación homogénea, particularmente:

$$T_{tcp}^{base}, T_0^{base}, T_{tcp}^6 \quad (5)$$

Al desacoplar la base y la herramienta final se puede conocer:

$$T_6^0 = [T_0^{base}]^{-1} T_{tcp}^{base} [T_{tcp}^6]^{-1} \quad (6)$$

También se sabe que:

$$T_6^0 = T_1^0 T_2^1 T_3^2 T_4^3 T_5^4 T_6^5 \quad (7)$$

En esta ecuación hay una alta complejidad en el lado derecho (RHS) y una baja complejidad en el lado izquierdo (LHS), por lo que se equilibra de la siguiente manera:

$$[T_1^0]^{-1} T_6^0 [T_6^5]^{-1} = T_2^1 T_3^2 T_4^3 T_5^4 \quad (8)$$

De forma genérica la matriz de transformación homogénea que vincula al sistema $i - 1$ con el sistema i es:

$$T_i^{i-1} = \begin{bmatrix} c_i & -s_i c_j & s_i s_j & a_i c_i \\ s_i & c_i c_j & -s_i s_j & a_i s_i \\ 0 & s_j & c_j & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9)$$

Donde:

$$\begin{aligned} s_i &:= \sin(\theta_i) \\ c_i &:= \cos(\theta_i) \\ s_j &:= \sin(\alpha_i) \\ c_j &:= \cos(\alpha_i) \end{aligned} \quad (10)$$

NOTA: El subíndice i y j no indican sistemas diferentes, sino que i hace referencia al ángulo θ y j al ángulo α para simplificar la nomenclatura de las funciones.

Cada matriz de transformación homogénea está compuesta por cuatro partes:

$$[T_i^{i-1}] = \begin{bmatrix} R_{3 \times 3} & p_{3 \times 1} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se puede obtener la matriz de transformación homogénea inversa mediante la siguiente expresión:

$$\begin{aligned} [T_i^{i-1}]^{-1} &= \begin{bmatrix} R^T & -R^T p \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ R^T &= \begin{bmatrix} c_i & s_i & 0 \\ -s_i c_j & c_i c_j & s_j \\ s_i s_j & -c_i s_j & c_j \end{bmatrix} \\ p &= \begin{bmatrix} a_i c_i \\ a_i s_i \\ d_i \end{bmatrix} \\ -R^T p &= \begin{bmatrix} -c_i & -s_i & 0 \\ +s_i c_j & -c_i c_j & -s_j \\ -s_i s_j & +c_i s_j & -c_j \end{bmatrix} \begin{bmatrix} a_i c_i \\ a_i s_i \\ d_i \end{bmatrix} = \begin{bmatrix} -a_i \\ -d_i s_j \\ -d_i c_j \end{bmatrix} \end{aligned}$$

Y se obtiene la matriz de transformación homogénea inversa:

$$[T_i^{i-1}]^{-1} = \begin{bmatrix} c_i & s_i & 0 & -a_i \\ -s_i c_j & c_i c_j & s_j & -d_i s_j \\ s_i s_j & -c_i s_j & c_j & -d_i c_j \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se toma esta plantilla de matriz y la plantilla de la matriz 9; se reemplazan valores y se simplifican para formar las matrices de transformación homogéneas presentes en la ecuación 8.

$$[T_1^0]^{-1} = \begin{bmatrix} c_1 & s_1 & 0 & 0 \\ 0 & 0 & -1 & d_1 \\ -s_1 & c_1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_6^5]^{-1} = \begin{bmatrix} c_6 & s_6 & 0 & 0 \\ -s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_2^1] = \begin{bmatrix} s_2 & c_2 & 0 & a_2 s_2 \\ -c_2 & s_2 & 0 & -a_2 c_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_3^2] = \begin{bmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_4^3] = \begin{bmatrix} -s_4 & 0 & c_4 & 0 \\ c_4 & 0 & s_4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_5^4] = \begin{bmatrix} c_5 & 0 & -s_5 & 0 \\ s_5 & 0 & c_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_6^0] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La ecuación 8, con sus matrices ya determinadas, se carga en un script de MATLAB `sym_1.m` para, mediante trabajo simbólico, determinar la expresión final simplificada de cada lado de la ecuación; se obtiene:

$$\text{LHS} = \begin{bmatrix} c_6(r_{11}c_1 + r_{21}s_1) - s_6(r_{12}c_1 + r_{22}s_1) & c_6(r_{12}c_1 + r_{22}s_1) + s_6(r_{11}c_1 + r_{21}s_1) & r_{13}c_1 + r_{23}s_1 & xc_1 + ys_1 \\ r_{32}s_6 - r_{31}c_6 & -r_{32}c_6 - r_{31}s_6 & -r_{33} & d_1 - z \\ c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) & c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) & r_{23}c_1 - r_{13}s_1 & yc_1 - xs_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{RHS} = \begin{bmatrix} c_5c_{234} & -s_{234} & -s_5c_{234} & a_3s_{23} + a_2s_2 + d_5s_{234} \\ c_5s_{234} & c_{234} & -s_5s_{234} & -a_3c_{23} - a_2c_2 - d_5c_{234} \\ s_5 & 0 & c_5 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Es importante resaltar que los valores r_{ij} , x , y y z , son valores conocidos; son la información de entrada al cálculo de la cinemática inversa. También es necesario aclarar que $c_{abc} = \cos(a + b + c)$ y $s_{abc} = \sin(a + b + c)$.

Las ecuaciones formadas por los elementos de las matrices LHS y RHS son las siguientes:

$$c_6(r_{11}c_1 + r_{21}s_1) - s_6(r_{12}c_1 + r_{22}s_1) = c_5c_{234} \quad (11)$$

$$c_6(r_{12}c_1 + r_{22}s_1) + s_6(r_{11}c_1 + r_{21}s_1) = -s_{234} \quad (12)$$

$$r_{13}c_1 + r_{23}s_1 = -s_5c_{234} \quad (13)$$

$$xc_1 + ys_1 = a_3s_{23} + a_2s_2 + d_5s_{234} \quad (14)$$

$$r_{32}s_6 - r_{31}c_6 = c_5s_{234} \quad (15)$$

$$-r_{32}c_6 - r_{31}s_6 = c_{234} \quad (16)$$

$$-r_{33} = -s_5s_{234} \quad (17)$$

$$d_1 - z = -a_3c_{23} - a_2c_2 - d_5c_{234} \quad (18)$$

$$c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) = s_5 \quad (19)$$

$$c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) = 0 \quad (20)$$

$$r_{23}c_1 - r_{13}s_1 = c_5 \quad (21)$$

$$yc_1 - xs_1 = d_3 \quad (22)$$

Se toma la ecuación formada por la posición 3:4 (fila 3, columna 4, ecuación 22):

$$yc_1 - xs_1 = d_3 \quad (23)$$

$$yc_1 - xs_1 - d_3 = 0 \quad (24)$$

$$A = y, B = -x, C = -d_3 \quad (25)$$

$$Ac_1 + Bs_1 + C = 0 \quad (26)$$

Se aplica la técnica de sustitución por la tangente de medio ángulo:

$$\begin{aligned} t &= \tan \frac{\theta_1}{2} \\ \cos(\theta_1) &= \frac{1 - t^2}{1 + t^2} \\ \sin(\theta_1) &= \frac{2t}{1 + t^2} \end{aligned} \quad (27)$$

$$A \frac{1 - t^2}{1 + t^2} + B \frac{2t}{1 + t^2} + C = 0 \quad (28)$$

$$A(1 - t^2) + 2Bt + C(1 + t^2) = 0 \quad (29)$$

$$A - At^2 + 2Bt + C + Ct^2 = 0 \quad (30)$$

$$t^2(C - A) + t(2B) + (C + A) = 0 \quad (31)$$

$$t_{1,2} = \frac{-2B \pm \sqrt{4B^2 - 4(C - A)(C + A)}}{2(C - A)} \quad (32)$$

De esta última ecuación se pueden determinar dos valores que conducen a dos valores de θ_1 mediante:

$$\theta_{1-1,2} = 2 \arctan(t_{1,2}) \quad (33)$$

La sustitución de medio ángulo $t = \tan(\theta/2)$ establece una relación biyectiva para $\theta \in (-\pi, \pi)$. De este modo, al recuperar el ángulo mediante $\theta = 2 \tan^{-1}(t)$, se extiende el rango de la función arco tangente y se evita la ambigüedad de cuadrante.

Una vez determinados ambos valores para q_1 , se analiza la ecuación formada por los miembros de la posición 3:2 (fila 3, columna 2, ecuación 20):

$$c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) = 0 \quad (34)$$

$$A = -(r_{22}c_1 - r_{12}s_1) \quad (35)$$

$$B = r_{21}c_1 - r_{11}s_1$$

$$Ac_6 + Bs_6 = 0 \quad (36)$$

$$t_6 = \frac{A}{-B} \quad (37)$$

$$\theta_6 = \text{atan2}(A, -B) \quad (38)$$

De esta última ecuación se obtiene un θ_6 por cada valor de θ_1 .

Se toma la ecuación 3:1 (fila 3, columna 1, ecuación 19) y la ecuación 3:3 (fila 3, columna 3, ecuación 21):

$$s_5 = c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) \quad (39)$$

$$c_5 = r_{23}c_1 - r_{13}s_1$$

$$t_5 = \frac{s_5}{c_5} \quad (40)$$

$$\theta_5 = \text{atan2}(s_5, c_5) \quad (41)$$

Para determinar correctamente los valores de θ_5 es fundamental mantener trazabilidad entre los resultados obtenidos para las variables articulares que intervienen. De esta manera se obtiene un valor de θ_5 por cada combinación de θ_1 y θ_6 .

Desde las ecuaciones formadas en las posiciones 1:3 (fila 1, columna 3, ecuación 13) y 2:3 (fila 2, columna 3, ecuación 17):

$$\begin{aligned} -s_5 c_{234} &= r_{13} c_1 + r_{23} s_1 \\ -s_5 s_{234} &= -r_{33} \end{aligned} \quad (42)$$

$$\begin{aligned} A = c_{234} &= \frac{r_{13} c_1 + r_{23} s_1}{-s_5} \\ B = s_{234} &= \frac{r_{33}}{s_5} \end{aligned} \quad (43)$$

Se continúa tomando las ecuaciones de las posiciones 1:4 (fila 1, columna 4, ecuación 14) y 2:4 (fila 2, columna 4, ecuación 18):

$$\begin{aligned} x c_1 + y s_1 &= a_3 s_{23} + a_2 s_2 + d_5 s_{234} \\ d_1 - z &= -a_3 c_{23} - a_2 c_2 - d_5 c_{234} \end{aligned} \quad (44)$$

Se reemplaza lo obtenido previamente en estas ecuaciones:

$$\begin{aligned} x c_1 + y s_1 &= a_3 s_{23} + a_2 s_2 + d_5 B \\ d_1 - z &= -a_3 c_{23} - a_2 c_2 - d_5 A \end{aligned} \quad (45)$$

Para simplificar, se define:

$$\begin{aligned} a &= -d_5 A + z - d_1 \\ b &= d_5 B - x c_1 - y s_1 \end{aligned} \quad (46)$$

Y queda de la siguiente forma:

$$\begin{aligned} a_3 c_{23} &= -a_2 c_2 + a \\ a_3 s_{23} &= -a_2 s_2 - b \end{aligned} \quad (47)$$

Al observar el lado izquierdo de cada ecuación, puede notarse que son las componentes de un vector, por lo tanto, puede simplificarse al elevar al cuadrado y sumar cada lado entre sí.

$$(a_3 c_{23})^2 + (a_3 s_{23})^2 = (-a_2 c_2 + a)^2 + (-a_2 s_2 - b)^2 \quad (48)$$

$$a_3^2 = a_2^2 c_2^2 - 2a_2 c_2 a + a^2 + a_2^2 s_2^2 + 2a_2 s_2 b + b^2 \quad (49)$$

$$a_3^2 = a_2^2 + a^2 + b^2 + 2a_2 s_2 b - 2a_2 c_2 a \quad (50)$$

Se define:

$$\begin{aligned} C &= a_2^2 + a^2 + b^2 - a_3^2 \\ D &= 2a_2 b \end{aligned} \quad (51)$$

$$E = -2a_2 a$$

$$D s_2 + E c_2 + C = 0 \quad (52)$$

Se aplica la sustitución por la tangente de medio ángulo, de forma análoga a lo realizado en el primer ángulo y se obtiene:

$$t^2(C - E) + t(2D) + (C + E) = 0 \quad (53)$$

$$t_{1,2} = \frac{-2D \pm \sqrt{4D^2 - 4(C - E)(C + E)}}{2(C - E)} \quad (54)$$

$$\theta_{2-1,2} = 2 \arctan(t_{1,2}) \quad (55)$$

Se reutilizan las ecuaciones de 47 para formar:

$$t_{23} = \frac{s_{23}}{c_{23}} = \frac{-a_2 s_2 - b}{-a_2 c_2 + a} \quad (56)$$

$$\theta_{23} = \arctan(t_{23}) \quad (57)$$

$$\theta_3 = \theta_{23} - \theta_2 \quad (58)$$

Se recuerda que debe mantenerse la trazabilidad de resultados para evitar cruzar soluciones. Finalmente, se reutilizan las ecuaciones de 43:

$$c_{234} = \frac{r_{13}c_1 + r_{23}s_1}{-s_5} \quad (59)$$

$$s_{234} = \frac{-r_{33}}{s_5}$$

$$t_{234} = \frac{r_{33}}{-(r_{13}c_1 + r_{23}s_1)} \quad (60)$$

$$\theta_{234} = \text{atan2}(r_{33}, -(r_{13}c_1 + r_{23}s_1)) \quad (61)$$

$$\theta_4 = \theta_{234} - \theta_2 - \theta_3 \quad (62)$$

Se deben tener en cuenta las mismas consideraciones sobre la trazabilidad de soluciones que se han mencionado previamente.

Se han obtenido todas las soluciones geométricas posibles (8) desde las cuales puede elaborarse un cuadro de referencia para seguir la traza de cada solución más claramente:

	q_1	q_2	q_3	q_4	q_5	q_6
Solución 1	q_1	q_2	q_3	q_4	q_5	q_6
Solución 2	q_1	q_2^I	q_3^I	q_4^I	q_5	q_6
Solución 3	q_1	q_2^{II}	q_3^{II}	q_4^{II}	q_5^I	q_6^I
Solución 4	q_1	q_2^{III}	q_3^{III}	q_4^{III}	q_5^I	q_6^I
Solución 5	q_1^I	q_2^{IV}	q_3^{IV}	q_4^{IV}	q_5^{II}	q_6^{II}
Solución 6	q_1^I	q_2^V	q_3^V	q_4^V	q_5^{II}	q_6^{II}
Solución 7	q_1^I	q_2^{VI}	q_3^{VI}	q_4^{VI}	q_5^{III}	q_6^{III}
Solución 8	q_1^I	q_2^{VII}	q_3^{VII}	q_4^{VII}	q_5^{III}	q_6^{III}

El robot a desarrollar cuenta con articulaciones rotativas con rango $[-2\pi, 2\pi]$, lo que permite realizar un giro completo en cada sentido. Esto permite una mayor flexibilidad en la generación de trayectorias, ya que existen múltiples configuraciones articulares equivalentes para alcanzar una misma pose del efector final. Esto provoca una *periodicidad en las soluciones*, es decir, se puede alcanzar el mismo punto de un círculo al girar 15° o -345° .

En primer lugar deben conocerse la cantidad de soluciones totales considerando periodicidad en articulaciones, dado que el rango total es $\pm k\pi$ donde $k = 2$, se duplicarán las soluciones por cada articulación que tenga periodicidad. En este caso, se consideran 6 articulaciones con periodicidad igual a 2 (NOTA: se considera que el rango de la última articulación es $\pm 2\pi$, en realidad es una articulación libre por lo que conduciría a infinitas soluciones), así:

$$\text{Cantidad total de soluciones} = \text{Cantidad de soluciones geométricas} \times 2^6 = 8 \times 2^6 = 512 \quad (63)$$

Cabe aclarar que estas 512 soluciones no son geoméricamente independientes entre sí, sino variantes *equivalentes* de las 8 soluciones geométricas originales: cada una se obtiene sumando o restando 2π a una o más articulaciones, dentro del rango permitido por sus límites, sin alterar la pose alcanzada por el efector final. Es decir, las 512 soluciones se agrupan en 8 familias de 64 configuraciones articulares que llevan al manipulador exactamente al mismo punto, difiriendo únicamente en el número de vueltas completas dado por cada articulación.

La metodología elegida para formar soluciones coherentes es:

1. Se define una matriz vacía cuyo tamaño corresponde al número total de soluciones, dado por: *Cantidad de soluciones geométricas* $\times 2^6$.
2. Se construye una matriz máscara binaria de tamaño 64×6 , cuyas filas codifican la selección entre el valor original de cada articulación (0) y su equivalente periódico (1), garantizando así la generación de todas las combinaciones posibles. Esto se itera para cada solución geométrica encontrada.
3. Para cada solución geométrica, se calcula un vector auxiliar que contiene las variantes periódicas de las articulaciones, sumando o restando 2π según lo permitan los límites articulares, evitando desbordes.
4. Finalmente, utilizando la matriz máscara, se completa la matriz de soluciones combinando los valores originales y periódicos correspondientes.

A continuación, se presentan imágenes de la forma genérica de cada matriz.

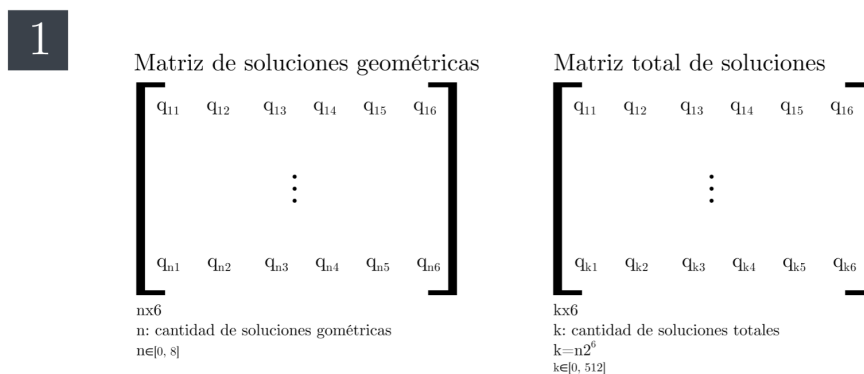


Figura 9: Matriz de soluciones geométricas y matriz total de soluciones

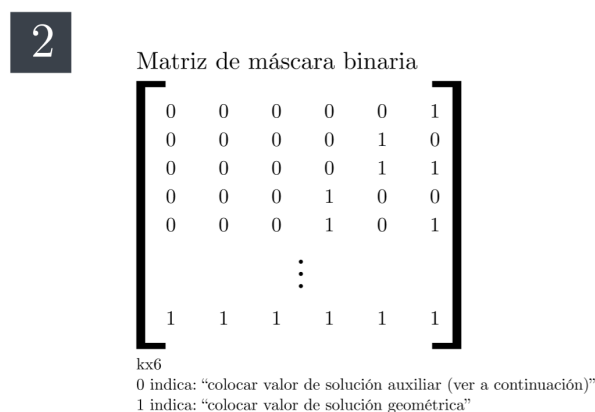


Figura 10: Matriz de máscara binaria

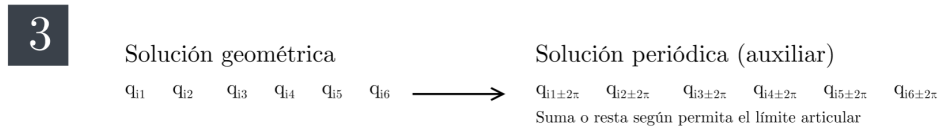


Figura 11: Solución geométrica genérica y cálculo de solución periódica correspondiente

La verificación de funcionamiento de la cinemática inversa se explica en A.1.

5.3. Convenciones de representación cartesiana

A lo largo del trabajo, las coordenadas cartesianas se expresan respecto del sistema de referencia $\{S_0\}$, ubicado en el centro de la superficie de la base que tiene contacto con la plataforma de apoyo. Las coordenadas (x, y, z) hacen referencia a la posición del TCP (*Tool Center Point*), mientras que la orientación se expresa mediante la convención *RPY* (Roll–Pitch–Yaw) respecto a los ejes de dicho sistema.

La convención RPY adoptada sigue el orden ZYX: las tres rotaciones son intrínsecas (sobre ejes móviles) y se aplican en el siguiente orden:

1. **Yaw** (γ): rotación en torno al eje z de $\{S_0\}$.
2. **Pitch** (β): rotación en torno al nuevo eje y , resultado de la rotación anterior.
3. **Roll** (α): rotación en torno al nuevo eje x , resultado de las dos rotaciones anteriores.

La matriz de rotación resultante se obtiene como el producto:

$$\mathbf{R} = \mathbf{R}_z(\gamma) \mathbf{R}_y(\beta) \mathbf{R}_x(\alpha). \quad (64)$$

Un ejemplo relevante para la tarea de transferencia es la configuración con el efector final apuntando hacia abajo. Dicha orientación se alcanza con una rotación de π en torno al eje y (Pitch) o en torno al eje x (Roll). En esta configuración, el valor del Yaw (γ) resulta independiente: cualquier rotación en torno al eje z mantiene el efector apuntando hacia abajo, dado que ese eje queda alineado con la vertical del sistema $\{S_0\}$.

5.4. Análisis y verificación de espacio de trabajo

Con el objetivo de verificar el espacio de trabajo del manipulador con foco en la tarea definida, se realiza un barrido de puntos en dos planos de interés: el plano x - y a la altura de las cajas, que permite evaluar el alcance en el plano horizontal donde se desarrolla la transferencia, y el plano x - z perpendicular a las cajas, que permite analizar la cobertura en profundidad necesaria para las maniobras de *picking* y *placement*.

Primera iteración: orientación fija

El barrido se realiza con un paso de 1 cm sobre las zonas de ambas cajas, con orientación de tarea fija (efector apuntando hacia abajo, $RPY \approx (0, 180, 0)$), verificando la alcanzabilidad de cada punto mediante la resolución de la cinemática inversa. Se distinguen dos tipos de barrido:

- **Vista superior (plano x - y):** se barren las zonas de ambas cajas — caja de origen ($x \in [0,12, 0,32]$ m, $y \in [0,075, 0,30]$ m) y caja de destino ($x \in [-0,32, -0,12]$ m, $y \in [0,075, 0,30]$ m) — para tres alturas representativas: $z = 0, 0,15$ y $0,25$ m.
- **Vista lateral (plano x - z):** se barren los mismos rangos en x de cada caja con $z \in [0, 0,25]$ m, para tres valores de profundidad: $y = 0,10, 0,15$ y $0,25$ m.

El resultado de esta primera iteración pone de manifiesto una limitación: con orientación fija, existen puntos dentro de la zona de la tarea que resultan inalcanzables por el manipulador. Esto puede observarse en la figura 12, correspondiente a la vista superior con $z = 0$, donde se identifican puntos no alcanzables en los bordes de las cajas.

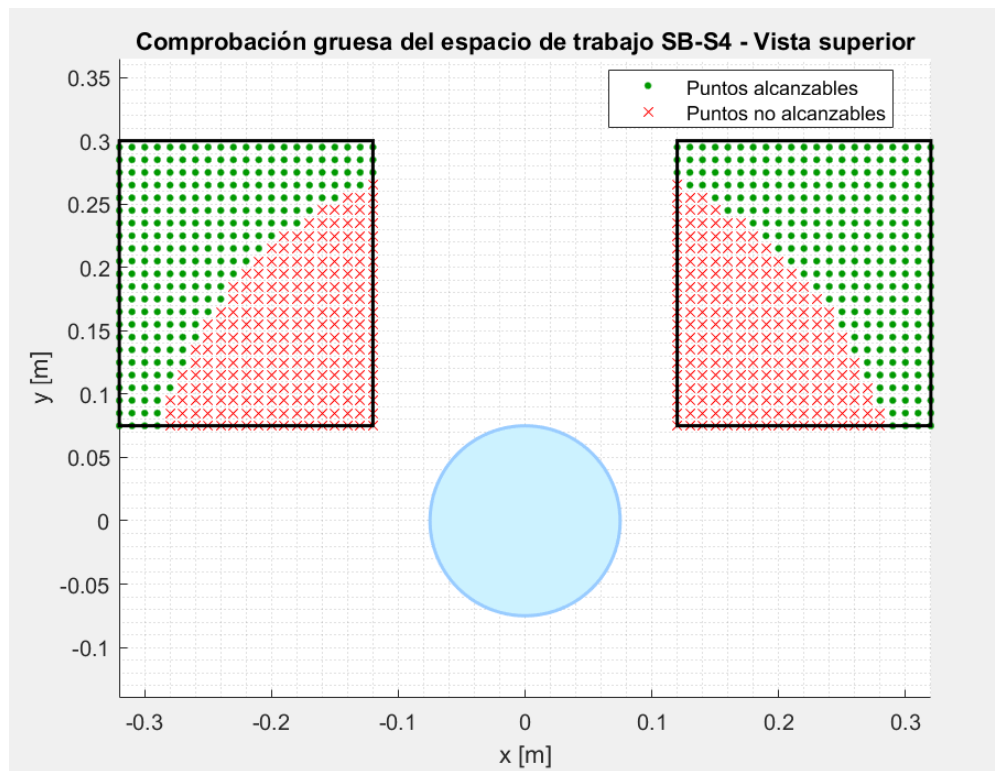


Figura 12: Vista superior del espacio de trabajo ($z = 0$ m) con orientación fija. Se identifican puntos inalcanzables en la zona de la tarea.

Este resultado requiere una solución: flexibilizar la orientación del efector final para ampliar el conjunto de configuraciones alcanzables en la zona de interés.

Segunda iteración: orientación aleatoria ante fallo

Se repite el mismo barrido con idénticos rangos y paso, manteniendo la orientación de tarea como primera opción. Cuando un punto no puede ser alcanzado con dicha orientación, se itera aleatoriamente sobre los tres ángulos RPY hasta un máximo de 20 intentos, verificando en cada uno si la cinemática inversa admite solución. El punto se clasifica como alcanzable si al menos uno de los intentos produce una solución válida.

El resultado es satisfactorio: la totalidad de los puntos de la zona de tarea resultan alcanzables. Las figuras 13 a 18 presentan los resultados para las vistas superior y lateral en todos los planos evaluados.

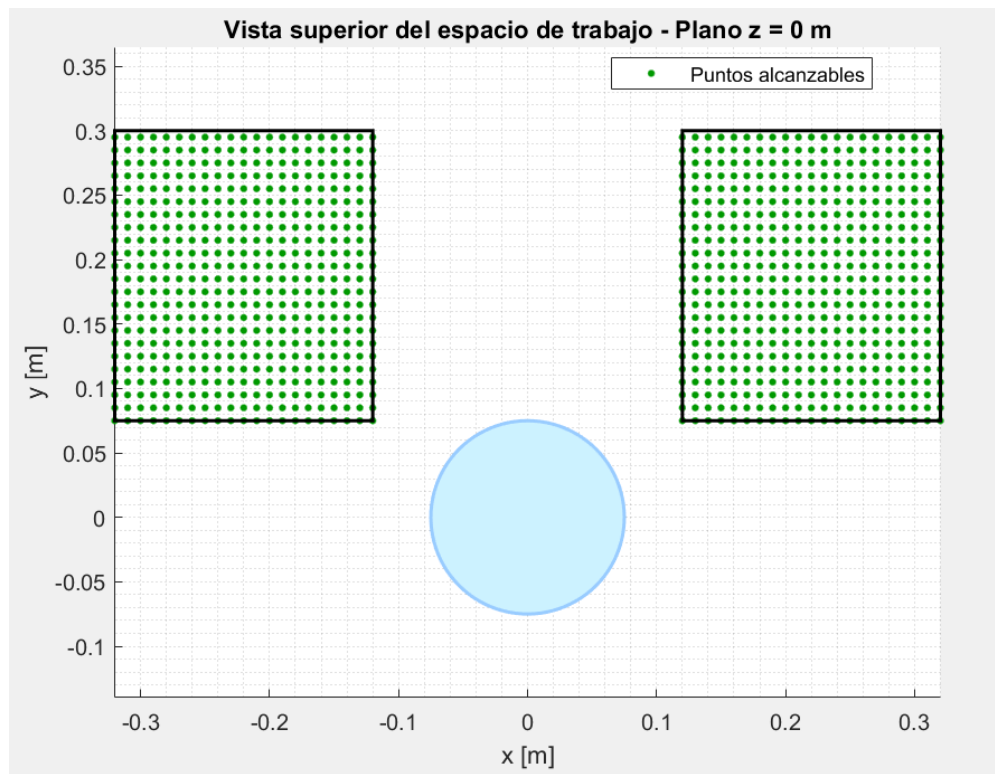


Figura 13: Vista superior del espacio de trabajo, $z = 0$ m.

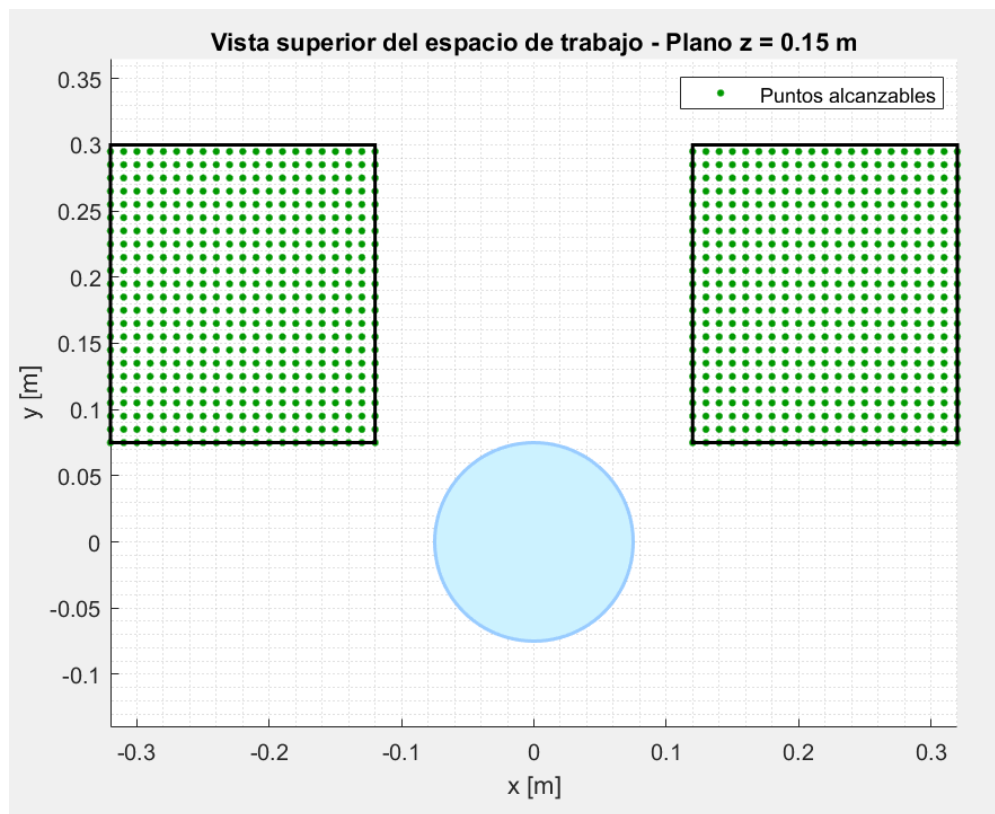


Figura 14: Vista superior del espacio de trabajo, $z = 0,15$ m.

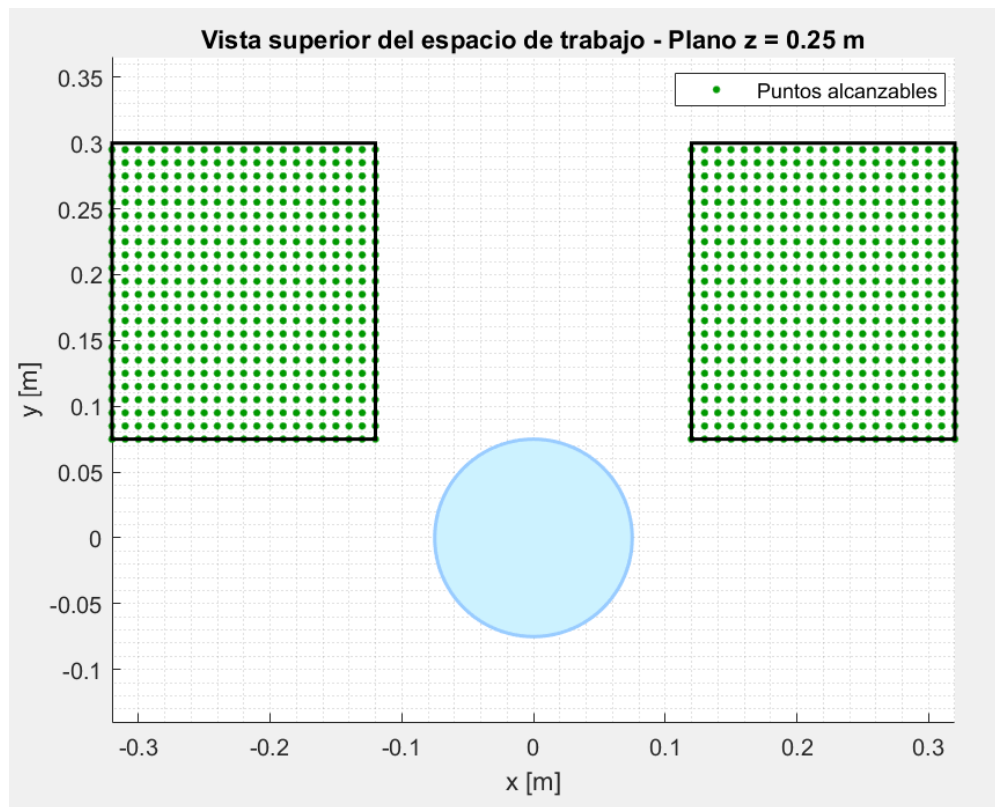


Figura 15: Vista superior del espacio de trabajo, $z = 0,25$ m.

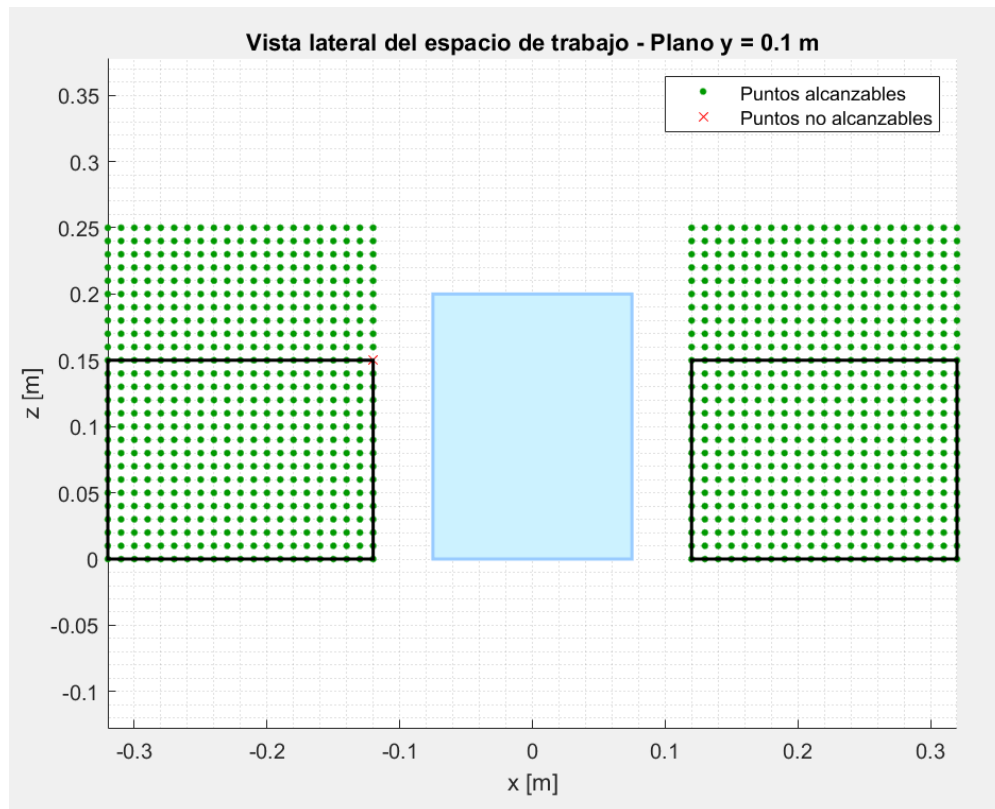


Figura 16: Vista lateral del espacio de trabajo, $y = 0,10$ m.

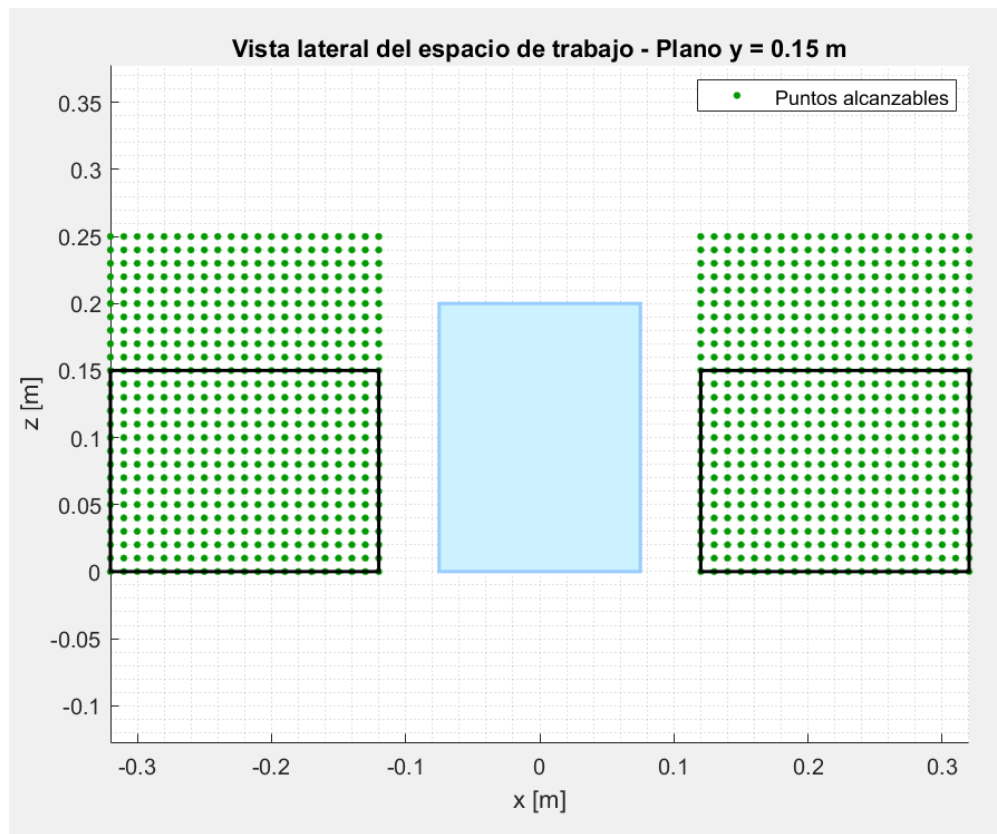


Figura 17: Vista lateral del espacio de trabajo, $y = 0,15$ m.

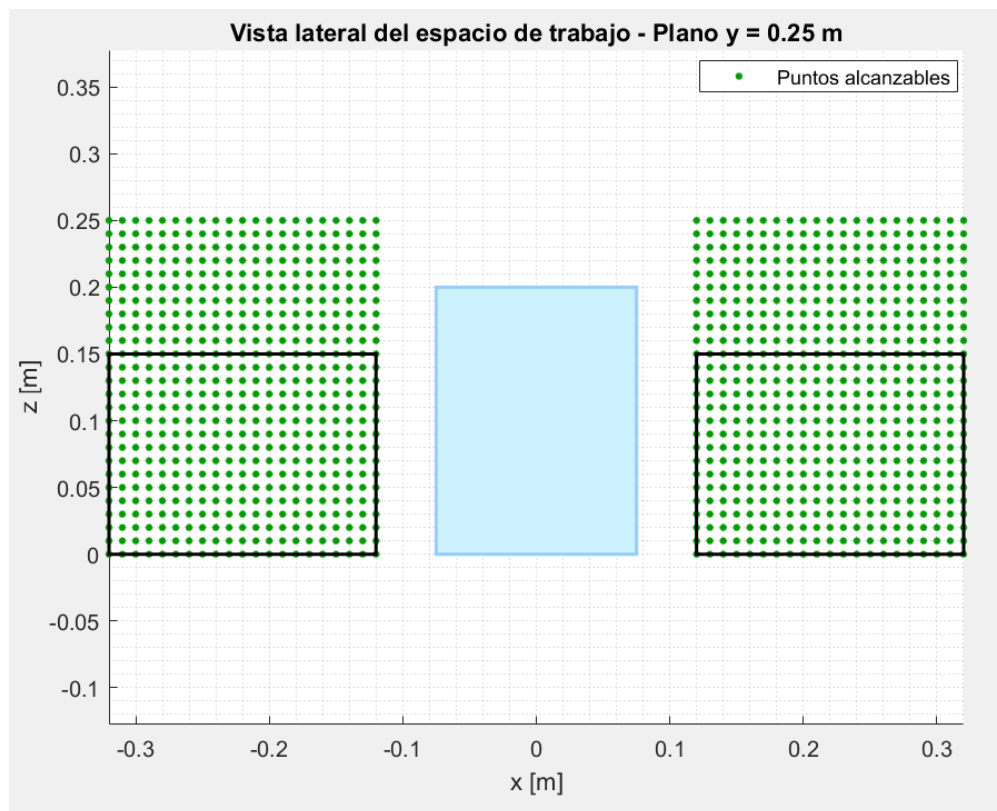


Figura 18: Vista lateral del espacio de trabajo, $y = 0,25$ m.

Se concluye que el manipulador es capaz de cubrir la totalidad de la zona de trabajo requerida por la tarea. Como trabajo futuro, queda pendiente verificar que las orientaciones alternativas encontradas durante el barrido aleatorio no generen colisiones entre el efector final y los bordes de las cajas, lo que requeriría incorporar restricciones geométricas en el proceso de selección de orientación.

5.5. Análisis de singularidades

Las singularidades representan configuraciones en las que un manipulador robótico pierde uno o más grados de libertad en el espacio cartesiano, independientemente del movimiento que se imponga en el espacio articular. En un robot de seis grados de libertad, estas configuraciones se identifican a partir del análisis del Jacobiano geométrico $\mathbf{J}(\mathbf{q})$, cuya pérdida de rango completo indica la presencia de una singularidad. Formalmente, una configuración $\mathbf{q} \in \mathbb{R}^6$ es singular si y solo si

$$\det(\mathbf{J}(\mathbf{q})) = 0 \quad (65)$$

En estas configuraciones, el robot exhibe dos consecuencias prácticas críticas: la imposibilidad de generar velocidades cartesianas en determinadas direcciones y la amplificación teóricamente ilimitada de velocidades articulares ante pequeñas velocidades cartesianas en las direcciones próximas al espacio nulo del Jacobiano. Por este motivo, la detección y el tratamiento de singularidades son requisitos indispensables en la generación de trayectorias y en el control de manipuladores industriales.

Al realizar un análisis geométrico, se identifican tres tipos de singularidades: de hombro, de codo y de muñeca. Matemáticamente, cada una corresponde a una condición específica sobre las variables articulares del robot.

- **Singularidad de muñeca:** ocurre cuando el eje de la articulación 4 y el eje de la articulación 6 se alinean, colapsando un grado de libertad en la orientación del efector final. La condición que la define es

$$q_5 = k\pi, \quad k \in \mathbb{Z}. \quad (66)$$

- **Singularidad de codo:** ocurre cuando los ejes de las articulaciones 2 y 4 se alinean, de modo que ambas articulaciones producen el mismo efecto rotacional y el Jacobiano pierde rango. La condición que la define es

$$q_3 = k\pi, \quad k \in \mathbb{Z}. \quad (67)$$

- **Singularidad de hombro:** ocurre cuando el punto de intersección de los ejes articulares 5 y 6 se encuentra sobre el plano que forman los ejes articulares 1 y 2. La forma analítica de esta explicación es hacer que la proyección vertical total de a_2 , a_3 y d_5 sea nula:

$$a_2 \sin(q_2) + a_3 \sin(q_2 + q_3) + d_5 \sin(q_2 + q_3 + q_4) = 0 \quad (68)$$

La verificación de las singularidades se explica en [A.2](#).

6. Generación de trayectorias

6.1. Generación de trayectorias

La generación de trayectorias constituye una etapa fundamental en el control de manipuladores robóticos, ya que permite definir la evolución temporal de las variables articulares o del efector final entre configuraciones iniciales y finales previamente establecidas. Este proceso no solo implica determinar el camino geométrico a seguir, sino también garantizar que el movimiento cumpla con restricciones físicas del sistema, tales como límites de velocidad, aceleración y suavidad.

El cálculo de la trayectoria se realiza dentro del script `gen_traj.m` el cual recibe los siguientes argumentos:

- **R**: definición del robot.
- **cpoints_input**: matriz de puntos vía en coordenadas cartesianas, uno por fila, con una séptima columna que indica el modo de interpolación para llegar a ese punto (0: espacio articular, 1: espacio cartesiano).
- **q0**: coordenada articular inicial del robot.
- **qdmax, qddmax**: velocidades y aceleraciones máximas articulares.
- **verbose**: controla la verbosidad de la cinemática inversa interna.
- **vmax, amax, wmax, alphamax**: velocidades y aceleraciones lineales y angulares máximas del efector final.

El trabajo comienza por analizar la séptima columna de cada fila (punto vía) del argumento **cpoints_input**, para determinar en qué modo de interpolación llegar a ese punto: 0 para interpolación en espacio articular y 1 para interpolación en espacio cartesiano.

Si el modo es 0, el primer paso es empaquetar todos los puntos vía consecutivos dentro de la matriz de *waypoints* para obtener una trayectoria suave y continua sin detención en cada punto vía dados originalmente. En segundo lugar, estos puntos se transforman al espacio articular mediante la aplicación del cálculo analítico de la cinemática inversa provisto en el script `inv_kinematics.m`. Los puntos vía en el espacio articular son enviados al interpolador de espacio articular provisto por el toolbox **mstraj**.

En el otro caso, cuando el modo es 1, la interpolación se da en el espacio cartesiano por lo que simplemente se envía el punto objetivo al interpolador en espacio cartesiano provisto en el toolbox *ctray* en conjunto con la ley de movimiento definida en *s*. La ley de movimiento se calcula de modo que el tiempo total del movimiento quede definido por el parámetro más comprometido (*vmax*, *amax*, *wmax*, *alphamax*).

En ambos casos, al calcular la trayectoria, esta se acopla a continuación de la trayectoria ya definida en una matriz. El último elemento de esta matriz se utiliza como punto inicial de la siguiente interpolación, en el caso de que esté vacía, es decir, que sea la primera interpolación, se utilizará el argumento **q0**.

La validación del funcionamiento del módulo de generación de trayectorias se presenta en el Apéndice A.3.

6.2. Escalamiento de velocidades y aceleraciones

Una vez generada una trayectoria preliminar, es necesario verificar que el perfil de velocidades y aceleraciones resultante respete los límites cinemáticos del robot, tanto en el espacio articular como en el espacio cartesiano. Para esto se realiza el siguiente procedimiento de dos etapas.

Primera etapa: generación preliminar y cómputo del factor de escala

Se genera la trayectoria con los parámetros nominales. A continuación, se recurre a la función auxiliar `compute_scale`, la cual calcula numéricamente las velocidades y aceleraciones articulares mediante diferencias finitas y, a través del Jacobiano del robot $\mathbf{J}(\mathbf{q})$, obtiene las velocidades y aceleraciones cartesianas:

$$\mathbf{v} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}, \quad \mathbf{a} = \mathbf{J}(\mathbf{q}) \ddot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q}) \dot{\mathbf{q}}. \quad (69)$$

Con estos valores se determina si algún límite es superado y en qué medida. La clave del método reside en la relación entre el factor de escala temporal k y sus efectos sobre la cinemática: si se ejecuta el mismo recorrido en k veces más tiempo, las velocidades se reducen en un factor $1/k$ y las aceleraciones en $1/k^2$. Por esto, los factores de escala individuales se calculan de forma diferente según se trate de una restricción de velocidad o de aceleración:

$$k_{\dot{q}} = \max_i \frac{|\dot{q}_i|_{\max}}{\dot{q}_{i,\max}}, \quad k_{\ddot{q}} = \sqrt{\max_i \frac{|\ddot{q}_i|_{\max}}{\ddot{q}_{i,\max}}}, \quad (70)$$

y de manera análoga para las magnitudes cartesianas:

$$k_v = \frac{\|\mathbf{v}\|_{\max}}{v_{\max}}, \quad k_\omega = \frac{\|\boldsymbol{\omega}\|_{\max}}{\omega_{\max}}, \quad k_a = \sqrt{\frac{\|\mathbf{a}\|_{\max}}{a_{\max}}}, \quad k_\alpha = \sqrt{\frac{\|\boldsymbol{\alpha}\|_{\max}}{\alpha_{\max}}}. \quad (71)$$

La raíz cuadrada en los factores de aceleración refleja que, al estirar el tiempo por k , la aceleración se reduce en k^2 , por lo que para cancelar una violación de factor r en aceleración basta con $k = \sqrt{r}$. El factor de escala final del segmento es:

$$k = \max(1, k_{\dot{q}}, k_{\ddot{q}}, k_v, k_\omega, k_a, k_\alpha). \quad (72)$$

Si $k = 1$ no existe ninguna violación y la trayectoria preliminar es válida.

Segunda etapa: regeneración de la trayectoria

Si $k > 1$, se regenera la trayectoria con el tiempo extendido. En el caso de interpolación articular (**mstraj**), esto equivale a reducir los límites de velocidad articular a $\dot{\mathbf{q}}_{\max}/k$ y extender el tiempo de rampa a $t_{\text{acc}} \cdot k$, con lo que la trayectoria resultante cumple todos los límites. En el caso de interpolación cartesiana, simplemente se extiende el tiempo total de t_f a $k \cdot t_f$ y se regenera la ley de movimiento quintic. En este modo existe además una estimación inicial de t_f basada en las propiedades analíticas del perfil quintic empleado, que sirve como punto de partida para la verificación posterior.

El resultado del escalamiento aplicado, incluyendo la comparativa entre los perfiles de velocidad antes y después del proceso tanto en el espacio articular como en el cartesiano, se presenta en el Apéndice A.3 (figuras 47 y 48).

7. Diseño y adaptación de CAD

El diseño tridimensional del robot constituye una etapa fundamental para la visualización y validación del modelo cinemático desarrollado. Con este objetivo, se elaboraron los modelos CAD de cada eslabón en el entorno *Fusion 360*, exportándolos posteriormente en formato STL para su integración en el entorno de simulación. Dado que las dimensiones y unidades de los archivos exportados no coinciden directamente con los parámetros del modelo DH, se desarrolló el script `scale.m`, que se encarga de escalar cada STL de forma que las dimensiones coincidan con las definidas en el modelo DH. Adicionalmente, durante el proceso de diseño se buscó maximizar el aprovechamiento del rango articular de cada articulación, procurando que la configuración mecánica permita explotar al máximo las capacidades de movimiento del robot sin incurrir en interferencias geométricas entre sus componentes.

7.1. Visualización del modelo en MATLAB

La figura 19 muestra el ensamblaje completo del robot renderizado en *MATLAB* mediante la toolbox de *Peter Corke*, con los marcos de referencia DH superpuestos sobre cada eslabón y con transparencia aplicada sobre las mallas STL, lo que permite verificar visualmente la alineación entre los ejes articulares y la geometría de cada pieza. Se incluyen también los sistemas del modelo DH.

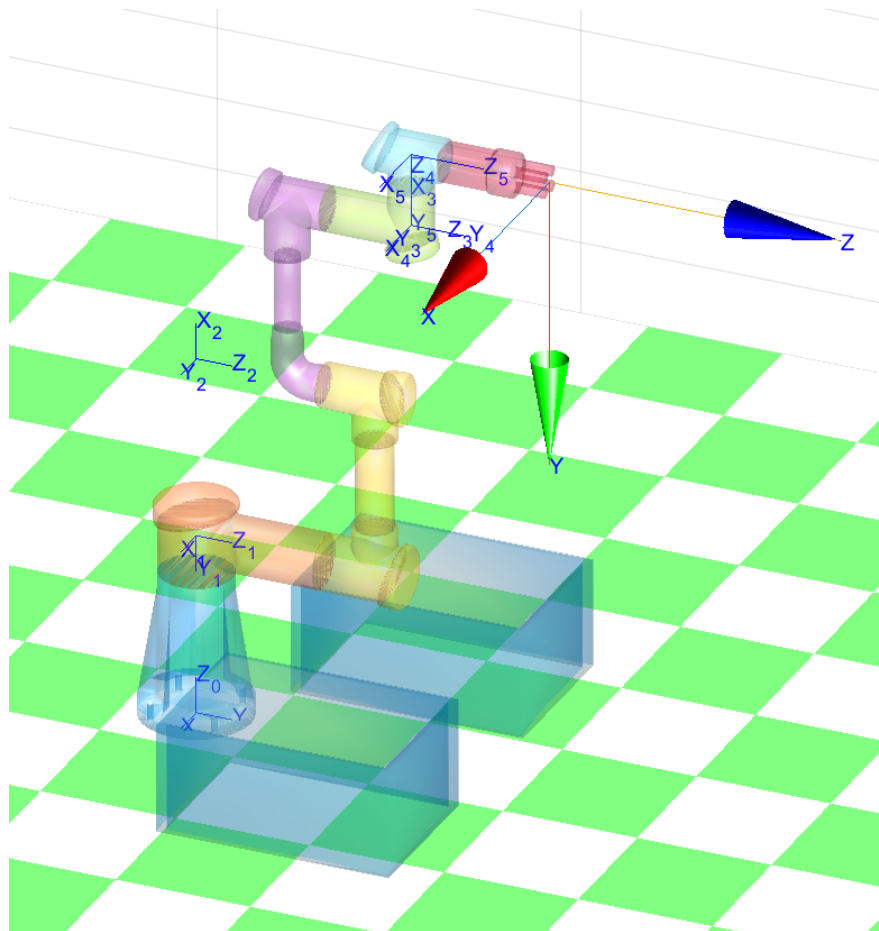


Figura 19: Modelo del robot renderizado en *MATLAB* con marcos de referencia DH.

7.2. Diseño de piezas en *Fusion 360*

A continuación se presentan los modelos de cada pieza diseñada en *Fusion 360*. Para cada componente se detallan las decisiones de diseño adoptadas en relación al aprovechamiento del rango articular y la evitación de interferencias geométricas. En todos los casos, el sistema de referencia de cada cuerpo

se adapta de forma que resulte coherente con el modelo DH, garantizando la coincidencia entre el eje de giro definido en la parametrización cinemática y el cuerpo sobre el cual dicho giro se produce.

7.2.1. Base

Para el diseño de la base se tienen en cuenta dos premisas. En primer lugar, se incrementa el grosor en la zona de apoyo con el objetivo de aumentar la estabilidad del conjunto. En segundo lugar, se incorpora una altura suficiente para posicionar al robot por encima de las cajas, tal como lo requiere la tarea descrita en la sección 3.

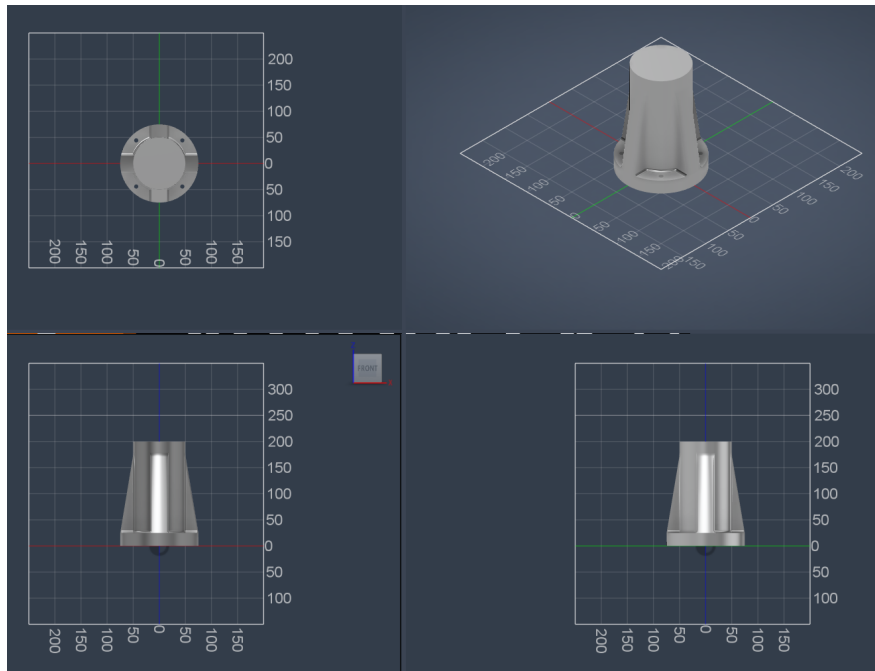


Figura 20: Diseño de la base en *Fusion 360*.

7.2.2. Eslabón 1

El eslabón 1 tiene como objetivo principal proveer una separación del eje central definido por la base, de forma que los eslabones posteriores cuenten con espacio suficiente para aprovechar su rango articular sin incurrir en interferencias geométricas.

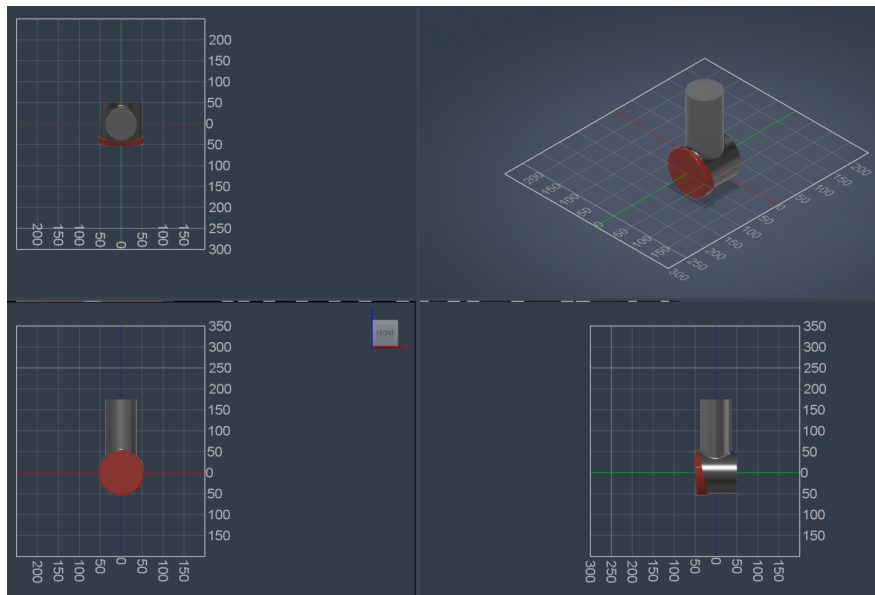


Figura 21: Diseño del eslabón 1 en *Fusion 360*.

7.2.3. Eslabón 2

El eslabón 2 cumple la misma función que el eslabón 1. La separación total requerida se distribuye entre ambos eslabones, logrando un equilibrio en la geometría del robot.

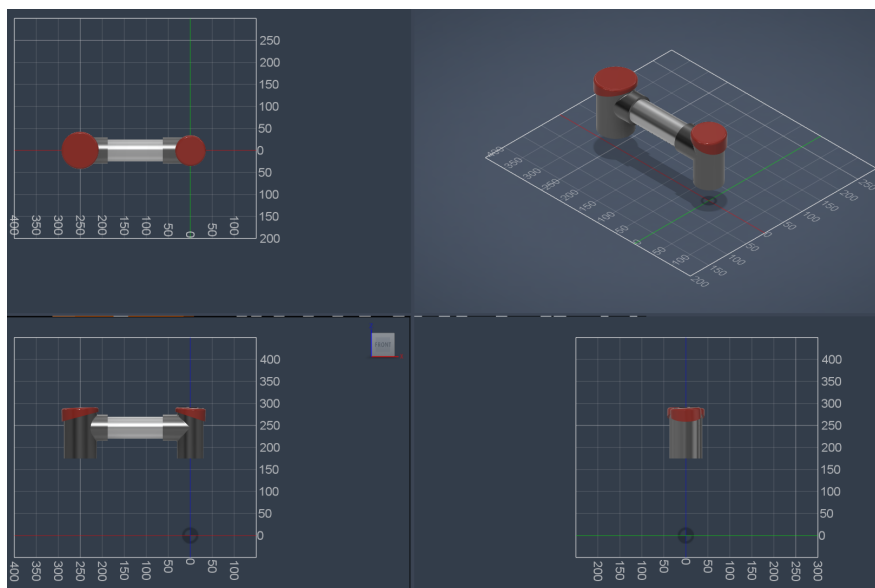


Figura 22: Diseño del eslabón 2 en *Fusion 360*.

7.2.4. Eslabón 3

El eslabón 3 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

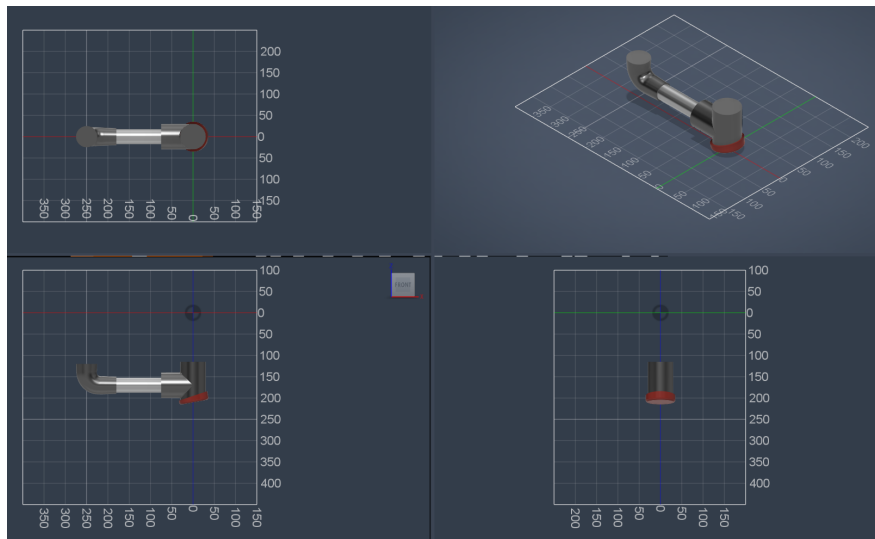


Figura 23: Diseño del eslabón 3 en *Fusion 360*.

7.2.5. Eslabón 4

El eslabón 4 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

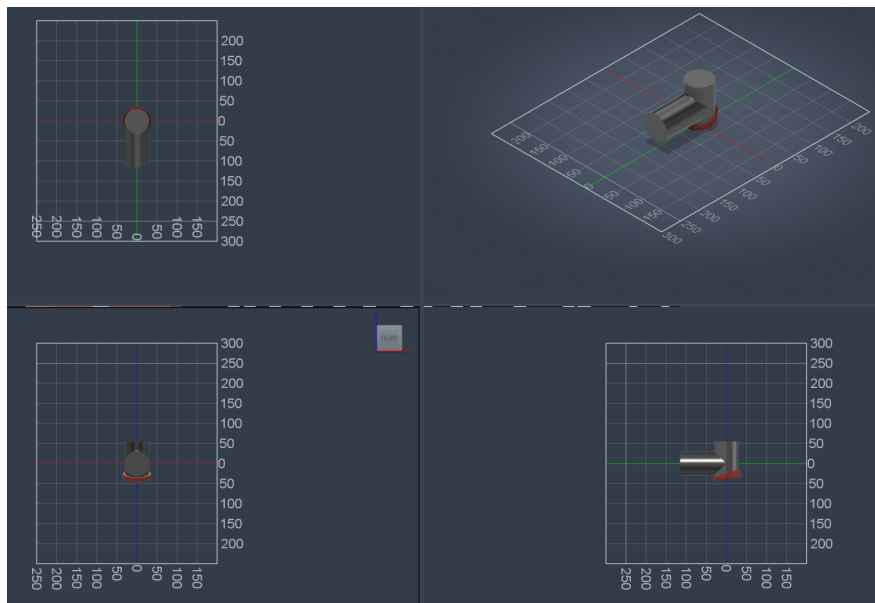


Figura 24: Diseño del eslabón 4 en *Fusion 360*.

7.2.6. Eslabón 5

El eslabón 5 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

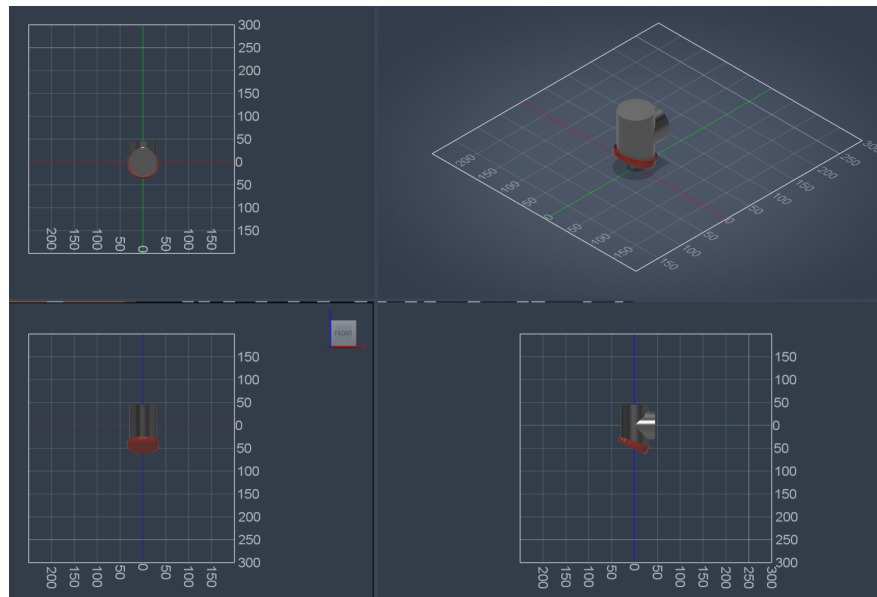


Figura 25: Diseño del eslabón 5 en *Fusion 360*.

7.2.7. Eslabón 6 —Herramienta

El eslabón 6 corresponde a la herramienta del sistema y no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

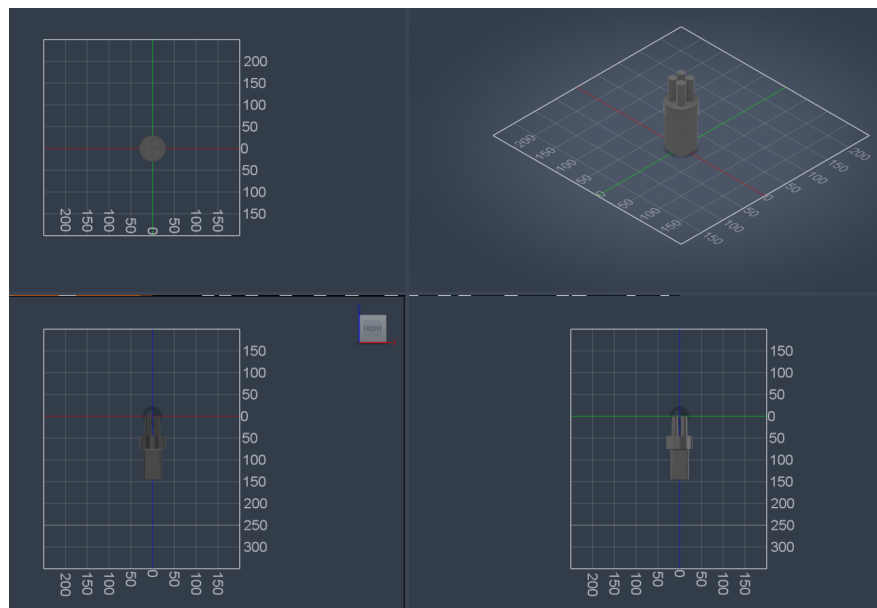


Figura 26: Diseño del eslabón 6 (herramienta) en *Fusion 360*.

7.3. Diseño alternativo: apertura unidireccional de eslabones

En robots de morfología similar al [UR3e](#), los eslabones se disponen de manera tal que, visto desde el plano lateral, el brazo adopta una forma en Z: el eslabón 2 se extiende en un sentido y el eslabón 3 se pliega en sentido opuesto, de forma que los giros de ambas articulaciones se compensan parcialmente entre sí. Esto implica que el aprovechamiento del rango articular de cada junta no es aditivo en la dirección de extensión del brazo.

Como alternativa, se diseñó una segunda versión del robot en la que todos los eslabones se abren en el mismo sentido de giro, adoptando una morfología en C vista desde el plano lateral. De este

modo, el aporte de cada articulación al alcance del efector final es aditivo, lo que permite un mejor aprovechamiento del rango articular disponible.

Cabe destacar que esta modificación es estrictamente geométrica: la parametrización cinemática del robot no varía, dado que la definición del modelo Denavit-Hartenberg depende de la disposición de los ejes articulares y no de la forma particular de los eslabones. En consecuencia, toda la cinemática directa e inversa desarrollada en la sección 5 es igualmente válida para ambos diseños.

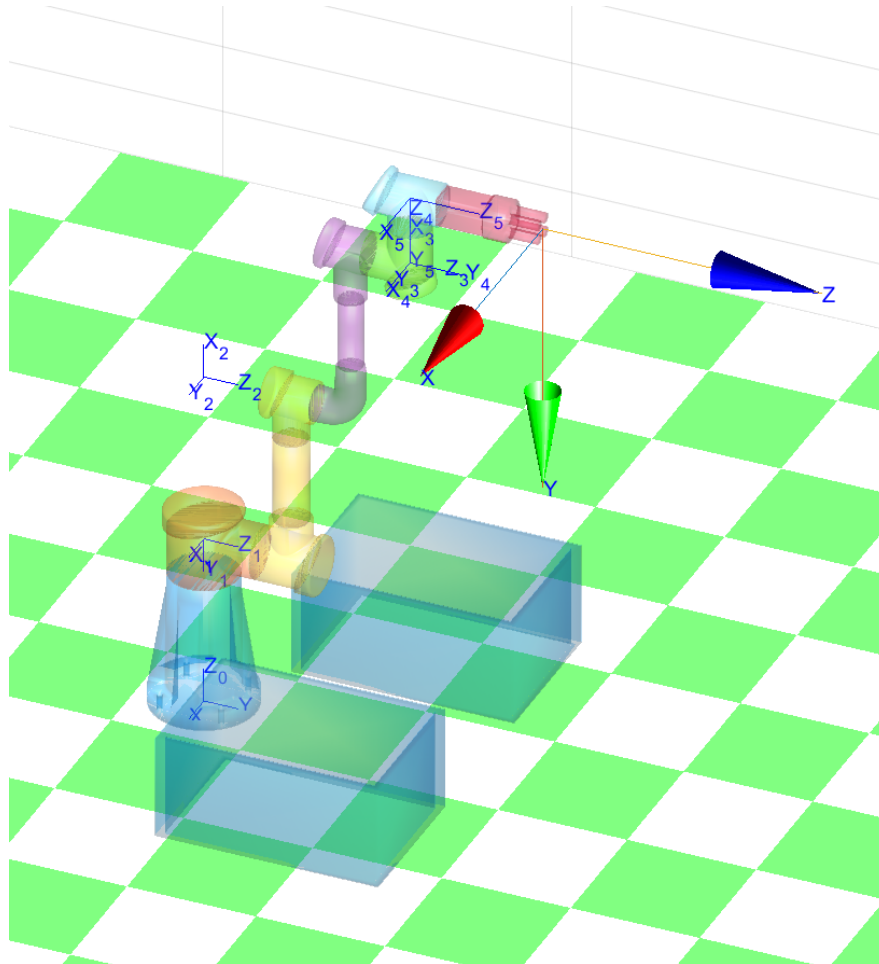


Figura 27: Modelo del robot con diseño alternativo de apertura unidireccional renderizado en *MATLAB*.

8. Simulación y verificación de tarea

A partir del cálculo conjunto de la cinemática inversa y el algoritmo de generación de trayectorias, puede desarrollarse la trayectoria de la tarea, la cual luego será simulada y analizada.

Para esto se definen los siguientes puntos vía, cuyas orientaciones corresponden a la convención RPY en grados (la herramienta apunta siempre hacia abajo, $\beta = 180^\circ$):

Punto	x [m]	y [m]	z [m]	α [°]	β [°]	γ [°]
caja_origen_superior	-0,22	0,30	0,20	0	180	0
caja_origen_inferior	-0,22	0,30	0,01	0	180	0
aux_1	-0,10	0,30	0,22	0	180	0
punto_medio	0,00	0,30	0,25	0	180	0
aux_2	0,10	0,30	0,22	0	180	0
caja_destino_superior	0,22	0,30	0,20	0	180	0
caja_destino_inferior	0,22	0,30	0,01	0	180	0

Cuadro 1: Puntos vía de la tarea de *bin-to-bin transfer*.

Nota: En el código se realiza la conversión de grados a radianes y, a los valores angulares, se les aplica una pequeña rotación adicional para evitar definir puntos vía que conduzcan a configuraciones singulares.

El *script* permite el comienzo desde un punto aleatorio o desde un punto conocido. Se recuerda que, en conjunto con los puntos vía, debe ser proporcionada la indicación del método de interpolación a utilizar; la secuencia es la siguiente:

1. **Posición inicial a punto medio:** Interpolación articular
2. **Punto medio a caja origen superior:** Interpolación articular
3. **caja origen superior a caja origen inferior:** Interpolación cartesiana
4. **Caja origen inferior a caja origen superior:** Interpolación cartesiana
5. **Caja origen superior a punto medio:** Interpolación articular
6. **Punto medio a caja destino superior:** Interpolación articular
7. **Caja destino superior a caja destino inferior:** Interpolación cartesiana
8. **Caja destino inferior a caja destino superior:** Interpolación cartesiana
9. **Caja destino superior a punto medio:** Interpolación articular

De este modo, se tendrá mayor control en el espacio cartesiano cuando se requiera ingresar y egresar de la caja de recogida y de colocación.

A continuación, se presenta una imagen luego de la simulación:

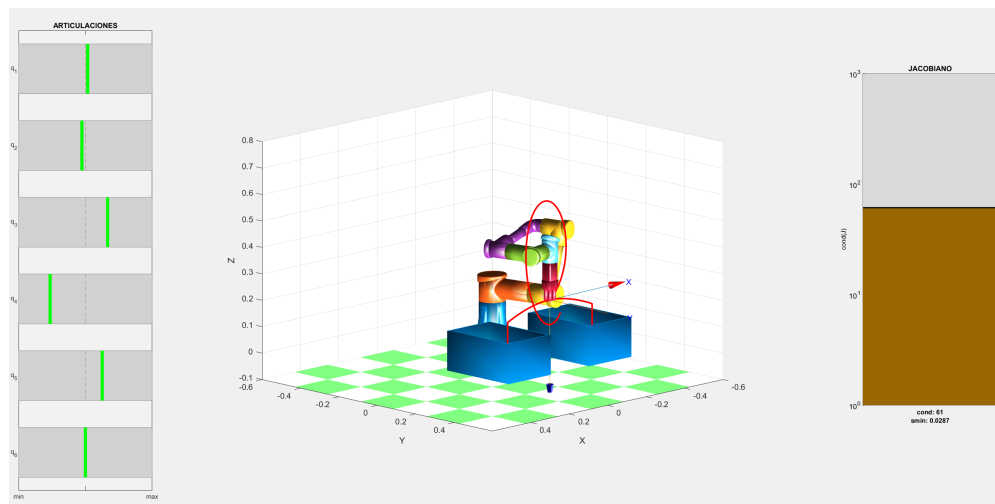


Figura 28: Imagen del final de la simulación de la tarea.

En la imagen puede verse el recorrido del robot, que ha dejado una traza roja correspondiente a la posición del efector final. Al inicio de la trayectoria se observa una maniobra de posicionamiento en el denominado punto medio, bajo una configuración articular favorable; esto mejora el comportamiento posterior al partir desde una posición alejada de los límites articulares. A los costados pueden verse gráficas animadas: a la izquierda, la posición instantánea de cada articulación, y a la derecha, el valor instantáneo del jacobiano, lo que permite evaluar posibles mejoras en la trayectoria generada.

Luego de la simulación, se obtienen las siguientes gráficas para el análisis de las variables cinemáticas del efector final y de las articulaciones. Puede observarse que los límites cinemáticos no se sobrepasan de forma significativa:

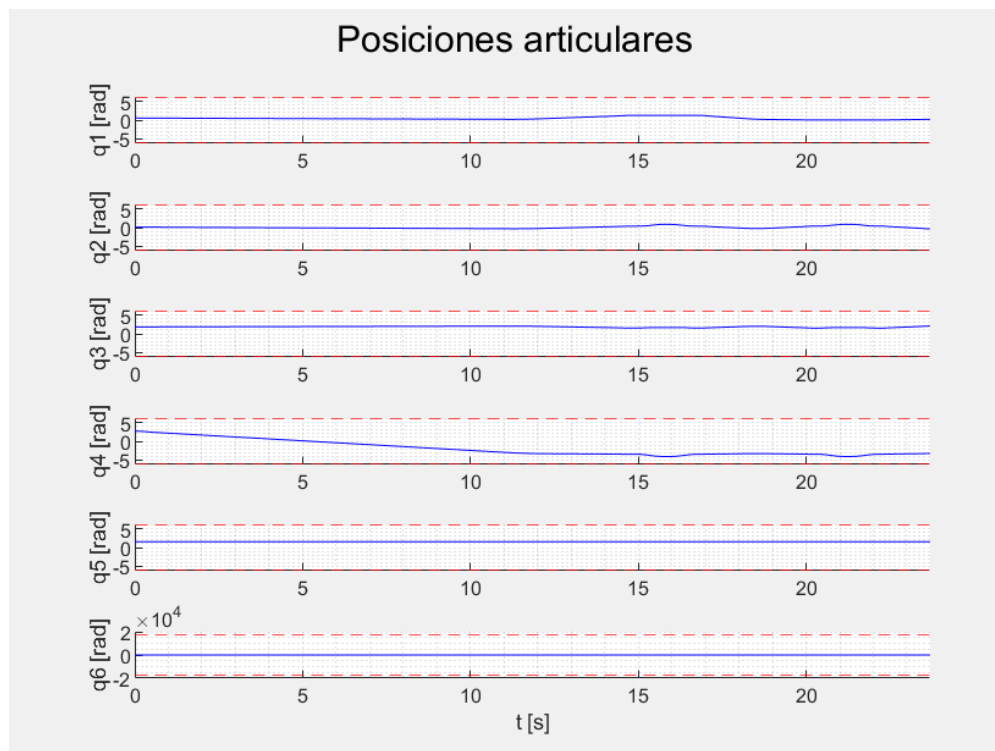


Figura 29: Evolución de posiciones articulares en la simulación de la tarea.

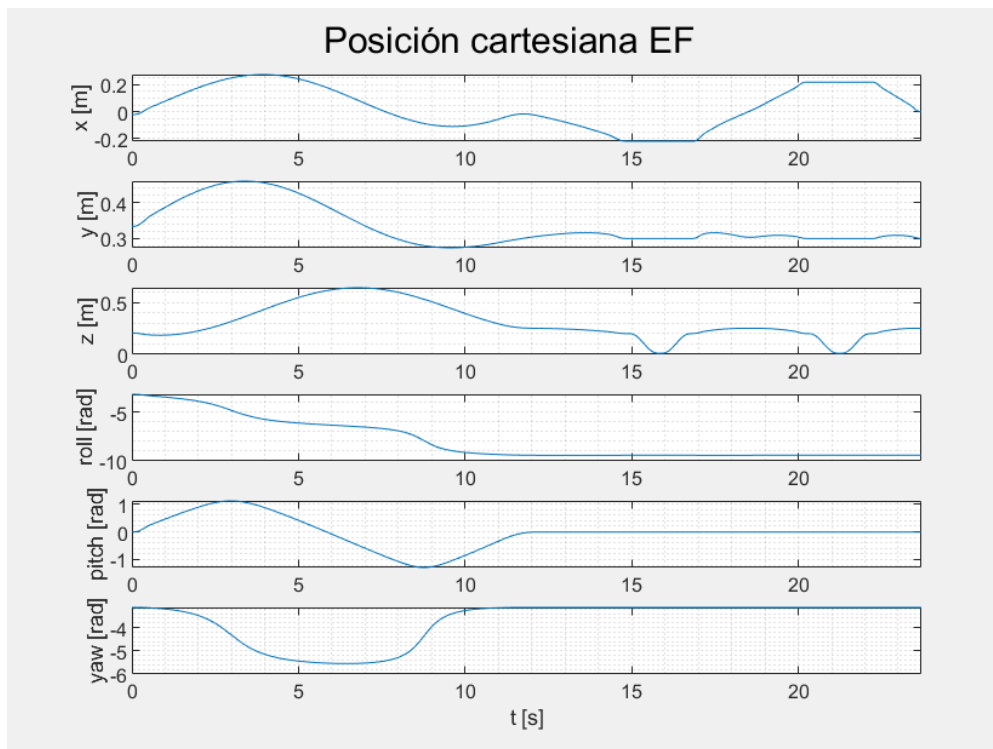


Figura 30: Evolución de la posición cartesiana del efector final en la simulación de la tarea.

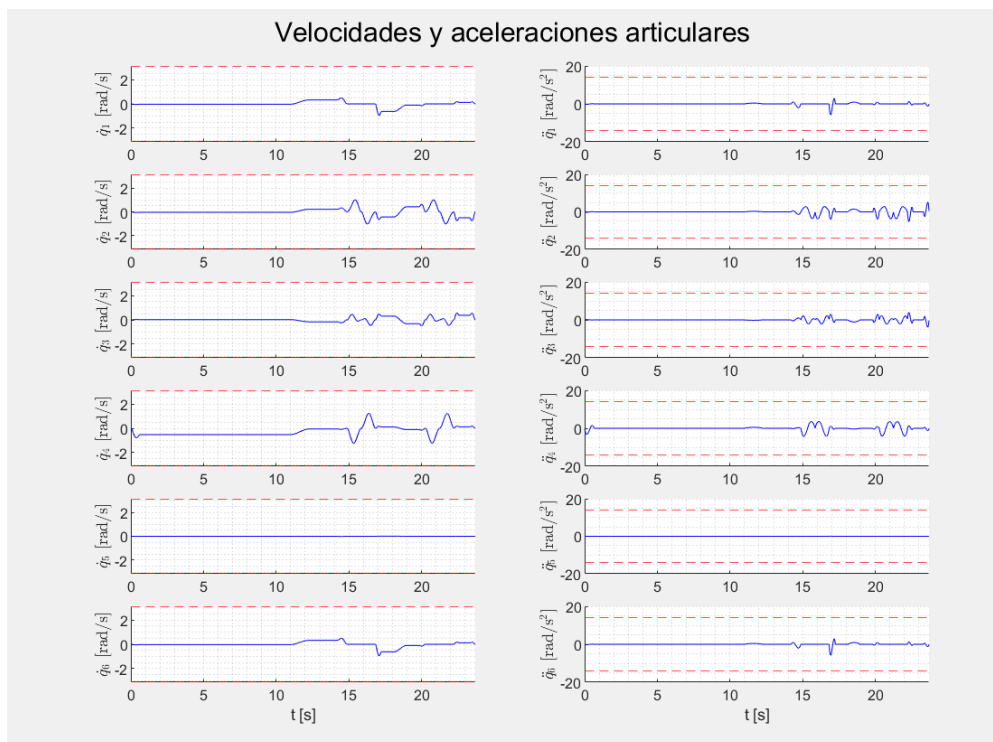


Figura 31: Evolución de velocidades y aceleraciones articulares en la simulación de la tarea.

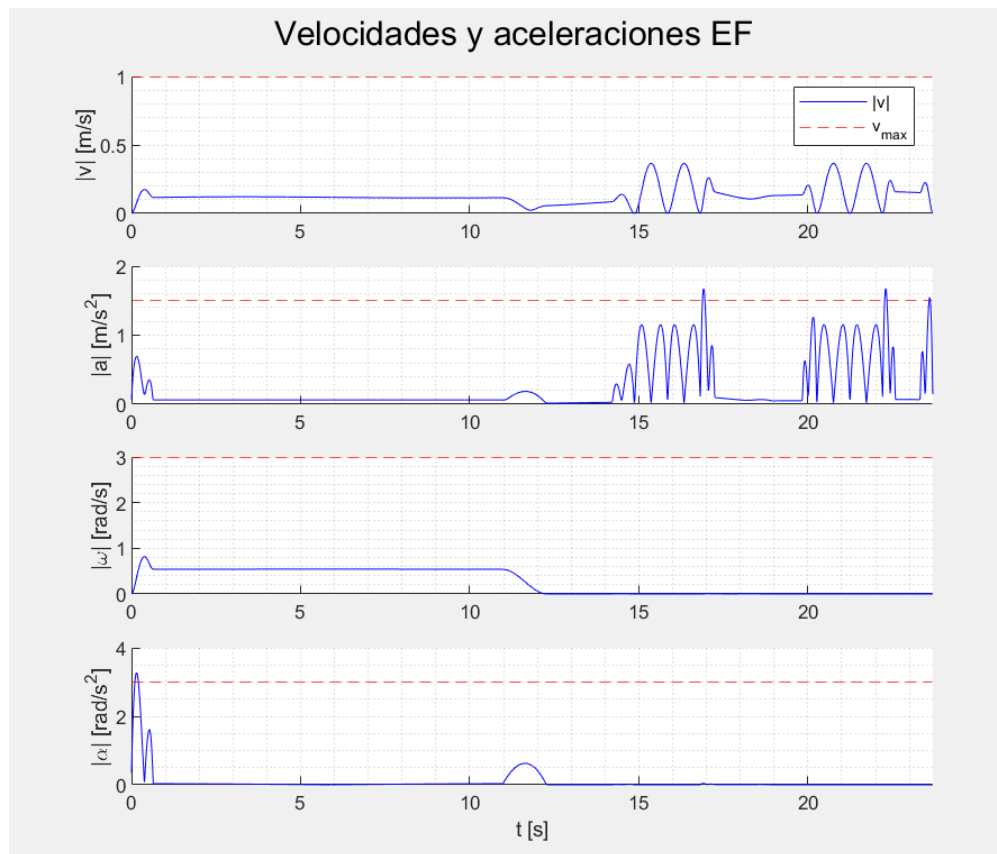


Figura 32: Evolución de velocidades y aceleraciones lineales y angulares del efector final en la simulación de la tarea.

9. Integración en entorno ROS 2

En esta sección se describe la integración del manipulador en el entorno *Robot Operating System 2* (ROS 2). El objetivo es trasladar el desarrollo realizado en MATLAB —cinemática directa, cinemática inversa y generación de trayectorias— a una plataforma de software orientada a sistemas robóticos reales, aportando modularidad, comunicación entre procesos y capacidad de simulación mediante RViz 2. En la búsqueda de maximizar el aprendizaje y personalización del sistema, se optó por desarrollar la mayoría de los nodos y módulos de forma manual.

La integración se estructura en torno a un cSonjunto de nodos que encapsulan responsabilidades bien definidas: cálculo cinemático, gestión de trayectorias, *teaching* y visualización a través de una interfaz gráfica. Estos nodos se comunican mediante tópicos y servicios de ROS 2, conformando una arquitectura distribuida que permite operar el robot de forma manual o autónoma.

El modelo del robot se describe en formato URDF (*Unified Robot Description Format*), derivado directamente de la parametrización Denavit-Hartenberg definida en la sección 5.1, lo que garantiza consistencia geométrica entre el modelo cinemático y su representación en el entorno de simulación.

9.1. Arquitectura del sistema

En primer lugar, se ha desarrollado un nodo `dr.launch.py` que se encarga de levantar todos los nodos y enviar, en la mayoría de los casos, los parámetros del robot `robot_params.yml`, el cual no contiene la definición URDF del robot sino parámetros de la cadena cinemática, límites cinemáticos y transformaciones de base y de herramienta.

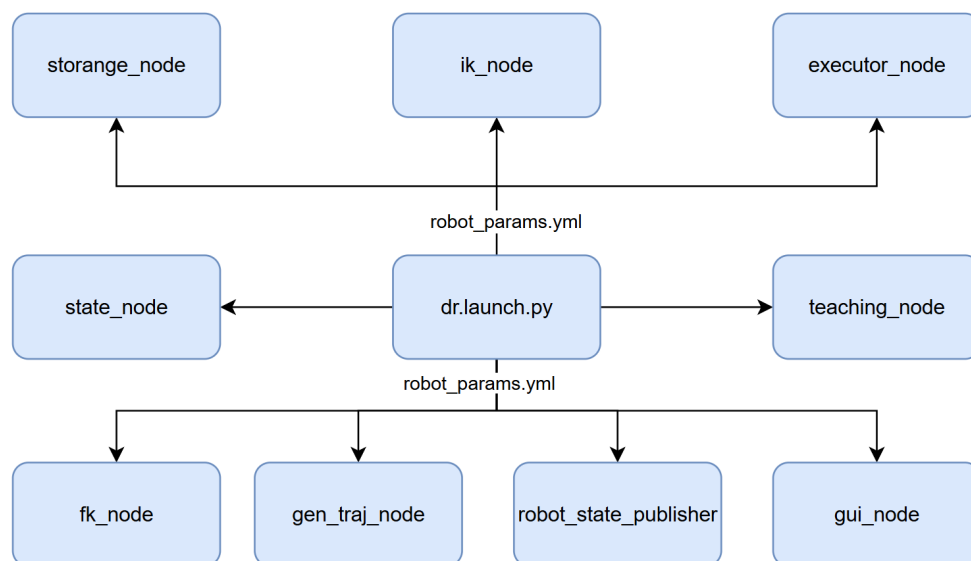


Figura 33: Esquema de levantamiento de nodos desde el nodo launcher.

A continuación se describe el propósito e interfaz principal de cada nodo. Los nodos de desarrollo propio se agrupan en paquetes con el prefijo `dr_`; el nodo `robot_state_publisher` es el nodo estándar de ROS 2 que publica las transformaciones del árbol cinemático a partir del URDF y del tópico `/joint_states`.

Cuadro 2: Nodos del sistema ROS 2 y sus interfaces principales.

Nodo	Paquete	Descripción e interfaz principal
fk_node	dr_kinematics	Resuelve la cinemática directa. Expone el servicio <code>/dr/solve_fk</code> : recibe los ángulos articulares $\mathbf{q}$ y devuelve la pose cartesiana del efector final.
ik_node	dr_kinematics	Resuelve la cinemática inversa analítica. Expone el servicio <code>/dr/solve_ik</code> : recibe una pose cartesiana y devuelve el conjunto de soluciones articulares válidas.
gen_traj_node	dr_trajectory	Genera trayectorias en espacio articular o cartesiano con escalado de velocidades. Expone <code>/dr/gen_traj</code> ; consume internamente <code>/dr/solve_ik</code> y <code>/dr/solve_fk</code> . Se suscribe a <code>/dr/robot_state</code> para obtener la configuración inicial.
state_node	dr_state	Agrega el estado articular (<code>/dr/joint_state</code>) y el estado de herramienta (<code>/dr/tool_state</code>), calcula la pose cartesiana o configuración articular actual mediante cinemática directa o inversa según corresponda y publica el estado unificado del robot en <code>/dr/robot_state</code> .
joint_state_bridge_node	dr_state	Convierte <code>/dr/robot_state</code> al mensaje estándar <code>sensor_msgs/JointState</code> publicado en <code>/joint_states</code> , necesario para la visualización en RViz 2 y para el <code>robot_state_publisher</code> .
executor_node	dr_executor	Ejecuta una trayectoria articular recibida como objetivo de la acción <code>/dr/execute</code> , publicando realimentación de posición en <code>/dr/joint_state</code> a lo largo del recorrido.
storage_node	dr_storage	Gestiona la persistencia de trayectorias en formato JSON. Expone los servicios <code>/dr/save_traj</code> , <code>/dr/load_traj</code> , <code>/dr/list_trajs</code> y <code>/dr/delete_traj</code> .
teaching_node	dr_teaching	Convierte incrementos cartesianos recibidos en <code>/dr/teaching_delta</code> en comandos de pose de herramienta publicados en <code>/dr/tool_state</code> , habilitando el movimiento manual incremental del efector final.
gui_node	dr_gui	Interfaz gráfica (PyQt5) que centraliza las funcionalidades del sistema: movimiento manual, enseñanza de puntos, generación y gestión de trayectorias, ejecución y análisis de variables cinemáticas. Se suscribe a <code>/dr/robot_state</code> , publica en <code>/dr/teaching_delta</code> y consume todos los servicios y la acción <code>/dr/execute</code> .

9.2. Interfaces personalizadas

Todas las interfaces de comunicación fueron definidas de forma propia en el paquete `dr_interfaces`. Se clasifican en mensajes, servicios y acciones.

En el caso de las trayectorias articulares, los mensajes de ROS 2 no admiten matrices bidimensionales de tamaño variable, por lo que se adoptó la convención de aplanarlas en vectores unidimensionales (`float64[]`). Cada grupo de seis valores consecutivos corresponde a una configuración articular $\mathbf{q} \in \mathbb{R}^6$, de modo que una trayectoria de N puntos se representa como un vector de $6N$ elementos. El campo entero `points` acompaña siempre a `q_traj` e indica el valor de N , necesario para reconstruir la matriz original al deserializar el mensaje.

Mensajes

Cuadro 3: Mensajes personalizados (dr_interfaces/msg).

Mensaje	Campos	Descripción
JointState	float64[6] q float64[6] qd float64[6] qdd	Estado articular completo: posición, velocidad y aceleración de las seis articulaciones.
RobotState	float64[6] q float64[6] qd float64[6] qdd float64[6] pose	Estado unificado del robot: estado articular más pose cartesiana del efector final $(x, y, z, \alpha, \beta, \gamma)$.
ToolState	float64[6] pose	Pose cartesiana de la herramienta $(x, y, z, \alpha, \beta, \gamma)$. Usada para comandar el movimiento del efector en modo <i>teaching</i> .
TeachingDelta	float64 dx, dy, dz float64 droll, dpitch, dyaw	Incremento cartesiano solicitado desde la GUI para el movimiento manual del efector final.

Servicios

Cuadro 4: Servicios personalizados (dr_interfaces/srv).

Servicio	Request	Response	Descripción
SolveFK	float64[6] q	float64[6] pose bool success	Cinemática directa.
SolveIK	float64[6] pose float64[6] q0 bool force	float64[6] q bool success	Cinemática inversa analítica.
GenTraj	float64[] pose int8 mode int8 speed_profile	float64[] q_traj int32 points bool success	Generación de trayectoria hacia una pose objetivo.
SaveTraj	string name float64[] q_traj int32 points	bool success	Persistencia de trayectoria en disco.
LoadTraj	string name	float64[] q_traj int32 points bool success	Carga de trayectoria desde disco.
ListTrajs	—	string[] names bool success	Lista de trayectorias almacenadas.
DeleteTraj	string name	bool success	Eliminación de trayectoria del disco.

Acciones

Cuadro 5: Acción personalizada (`dr_interfaces/action`).

Acción	Goal	Feedback	Result
Execute	float64[] q_traj int32 points	float64[6] q, qd, qdd int32 point_index float64 progress	bool success string message

9.3. Interrelación entre nodos

Los nodos del sistema se interrelacionan a través de cuatro flujos funcionales que operan de forma concurrente. A continuación se describe cada uno.

Flujo de estado (permanente). El `state_node` es el nodo central del sistema: agrega toda la información de posición del robot y la redistribuye de forma unificada. Recibe el estado articular desde `executor_node` a través del tópico `/dr/joint_state`, y los comandos de pose de herramienta desde `teaching_node` a través de `/dr/tool_state`. Con esta información, llama al servicio `/dr/solve_fk` del `fk_node` para calcular la pose cartesiana actual y publica el estado completo del robot en `/dr/robot_state`. Este tópico es consumido por `gui_node`, `teaching_node` y `gen_traj_node`. Paralelamente, el `joint_state_bridge_node` convierte `/dr/robot_state` al formato estándar `sensor_msgs/JointState` en `/joint_states`, que es consumido por el nodo desarrollado `robot_state_publisher` para mantener el árbol de transformaciones TF y habilitar la visualización en RViz 2.

Flujo de movimiento manual (*teaching*). Cuando el operador acciona los controles de movimiento manual en la GUI, el `gui_node` publica un incremento cartesiano en `/dr/teaching_delta`. El `teaching_node` recibe este incremento y, a partir de la pose actual disponible en `/dr/robot_state`, calcula la nueva pose objetivo y la publica en `/dr/tool_state`. El `state_node` recibe esta actualización, resuelve la cinemática inversa llamando a `/dr/solve_ik` del `ik_node` para obtener la configuración articular correspondiente, y publica el nuevo `/dr/robot_state`, cerrando el ciclo de realimentación hacia la GUI.

Flujo de generación de trayectoria. Cuando el operador solicita generar una trayectoria desde la GUI, el `gui_node` invoca el servicio `/dr/gen_traj` del `gen_traj_node`. Este último consulta `/dr/robot_state` para obtener la configuración articular inicial del robot, y luego llama iterativamente a `/dr/solve_ik` del `ik_node` y a `/dr/solve_fk` del `fk_node` para resolver las configuraciones articulares de cada punto vía y verificar la trayectoria generada. La trayectoria resultante, codificada como vector aplanado, es devuelta al `gui_node` en la respuesta del servicio.

Flujo de ejecución de trayectoria. Una vez disponible la trayectoria, el `gui_node` envía un objetivo (*goal*) a la acción `/dr/execute` del `executor_node`. Este recorre la trayectoria articular punto a punto, publicando en cada paso el estado articular actual en `/dr/joint_state`. El `state_node` procesa estos valores, actualiza `/dr/robot_state` y lo publica, de modo que tanto la GUI como RViz 2 reflejan el movimiento en tiempo real. Simultáneamente, el `executor_node` envía realimentación al cliente de la acción con la posición articular actual, el índice del punto en curso y el progreso normalizado de la ejecución.

Flujo de gestión de trayectorias almacenadas. El `gui_node` interactúa directamente con el `storage_node` a través de los servicios `/dr/save_traj`, `/dr/load_traj`, `/dr/list_trajs` y `/dr/delete_traj` para persistir, recuperar y administrar trayectorias en disco. El `storage_node` no tiene dependencias con ningún otro nodo del sistema, lo que garantiza su operación independiente.

9.4. Modelo URDF

El modelo del robot se describe en formato URDF (*Unified Robot Description Format*) y se aloja en el paquete `robot_description`. Su estructura fue derivada directamente de la parametrización Denavit-Hartenberg definida en la sección 5.1, garantizando consistencia geométrica entre ambas representaciones.

El árbol cinemático está compuesto por un eslabón raíz fijo (`world`), seis eslabones móviles (`link_1` a `link_6`) y un eslabón de herramienta (`tcp`), vinculados mediante seis articulaciones rotativas y una articulación fija. Cada articulación rotativa incorpora los parámetros de offset, traslación y rotación del eslabón correspondiente derivados de la tabla DH, de modo que la cadena cinemática resultante reproduce fielmente el modelo analítico. Las articulaciones 1 a 5 tienen límites de $\pm 2\pi$ rad ($\pm 360^\circ$); la articulación 6 se declara de tipo *continuous* en concordancia con el alcance libre definido en las especificaciones del robot.

El offset de herramienta (T_{tool}) se incorpora como una articulación fija entre `link_6` y `tcp`, con una traslación de 0.190 m en el eje Z, valor consistente con el parámetro `T_tool` de `robot_params.yml`.

Cada eslabón incluye una geometría visual asociada a la malla STL correspondiente del modelo CAD desarrollado en *Fusion 360* (sección 7). Los ejes de referencia de cada malla fueron alineados con los marcos DH durante el proceso de diseño, por lo que no fue necesario aplicar transformaciones adicionales entre el sistema de referencia cinemático y el visual.

Esto puede verse en la siguiente imagen correspondiente al entorno de simulación RViz 2:

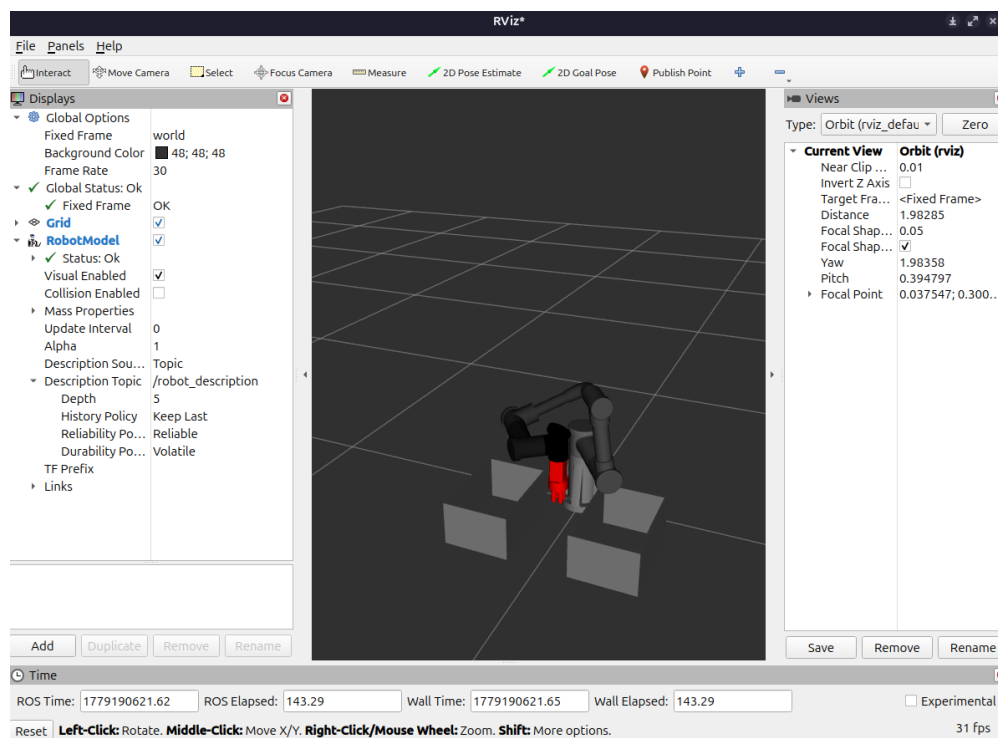


Figura 34: Modelo URDF visualizado en simulador RViz 2.

9.5. Cinemática

La cinemática directa e inversa se porta al entorno ROS 2 preservando los algoritmos analíticos desarrollados en la sección 5. Sin embargo, la *Robotics Toolbox* de Peter Corke, utilizada en MATLAB para operaciones de álgebra matricial homogénea y conversión de representaciones de orientación, no está disponible en el entorno Python de ROS 2.

Para resolver esta dependencia se implementó el módulo `transforms.py`, alojado en el paquete `dr_kinematics`. Este archivo replica las funciones del toolbox empleadas en el desarrollo, sin dependencias externas más allá de `numpy`:

- `transl(x, y, z)`: matriz de transformación homogénea de traslación pura.

- $\text{trotx}(\theta)$, $\text{troty}(\theta)$, $\text{trotz}(\theta)$: rotaciones elementales alrededor de los ejes X , Y y Z .
- $\text{inv_T}(\mathbf{T})$: inversa de una transformación homogénea, calculada eficientemente mediante R^T y $-R^T \mathbf{p}$.
- $\text{rot_to_rpy}(\mathbf{R})$: extrae los ángulos de Euler ZYX (roll, pitch, yaw) desde una matriz de rotación, equivalente a `tr2rpy` del toolbox.

Los parámetros cinemáticos del robot (tabla DH, límites articulares, transformaciones de base y herramienta) se cargan en tiempo de ejecución desde `robot_params.yml`, sin estar codificados en los nodos. Esto permite ajustar el modelo cinemático sin recompilar el código.

9.6. Generación de trayectorias

El módulo de generación de trayectorias se porta al nodo `gen_traj_node`, preservando los dos modos de interpolación: espacio articular (modo 0) y espacio cartesiano (modo 1), y el mecanismo de escalado de velocidades y aceleraciones descrito en la sección 6.2.

El servicio `/dr/gen_traj` acepta **una única pose cartesiana objetivo** por solicitud, junto con el modo de interpolación y un perfil de velocidad (lento, medio o rápido). Al recibir la solicitud, el nodo obtiene la configuración articular actual desde `/dr/robot_state` y genera la trayectoria desde esa configuración hasta el punto objetivo.

Esta decisión de diseño introduce una **limitación respecto a la implementación en MATLAB**: en `gen_traj.m`, la función recibía la lista completa de puntos vía y generaba una única trayectoria continua a través de todos ellos, aprovechando la capacidad de `mstraj` de interpolar sin detención en los puntos intermedios. En ROS 2, dado que el servicio acepta un solo punto por solicitud, cada llamada genera una trayectoria independiente con velocidad inicial y final nula. Como consecuencia, **el robot se detiene completamente en cada punto vía** antes de recibir la siguiente solicitud, lo que impide la interpolación fluida entre segmentos consecutivos.

Esta limitación podría resolverse en el futuro rediseñando el servicio para aceptar una lista de puntos vía en una única solicitud, replicando el comportamiento de `mstraj` en el entorno ROS 2.

9.7. Teaching

El modo *teaching* permite desplazar el efector final de forma incremental desde la interfaz gráfica, capturando configuraciones intermedias que luego se encadenan para construir una trayectoria. El mecanismo se divide en dos componentes: el nodo de *teaching* y la pestaña dedicada en la interfaz gráfica.

Nodo de teaching

El `teaching_node` actúa como un convertidor de desplazamientos relativos a poses absolutas. Al iniciarse, se suscribe a `/dr/robot_state` para obtener y actualizar continuamente la configuración articular actual $\mathbf{q}_0$ y la pose cartesiana correspondiente $\mathbf{c}_0 = [x, y, z, \alpha, \beta, \gamma]^\top$, donde α, β, γ son los ángulos de Euler ZYX.

Cuando la interfaz gráfica publica un incremento en `/dr/teaching_delta` — mensaje `TeachingDelta` con campos `dx`, `dy`, `dz`, `droll`, `dpitch`, `dyaw` — el nodo suma dicho incremento a $\mathbf{c}_0$ y publica la pose absoluta resultante en `/dr/tool_state`:

$$\mathbf{c}_{\text{objetivo}} = \mathbf{c}_0 + \Delta \mathbf{c}$$

Este enfoque garantiza que cada desplazamiento sea relativo a la pose *actual* del robot, independientemente de si el robot se ha movido entre pulsos de botón.

Pestaña de teaching en la GUI

La pestaña *Teaching* de la interfaz gráfica centraliza el flujo de trabajo de enseñanza de puntos. Su estructura es la siguiente:

- **Tabla de estado actual.** Muestra en tiempo real la configuración articular $\mathbf{q}$ (en radianes) y la pose cartesiana $[x, y, z, \alpha, \beta, \gamma]$ (α, β, γ : ángulos de Euler ZYX) del efector final. Los valores se actualizan cada vez que el nodo recibe un mensaje en `/dr/robot_state`.
- **Controles de jog.** Para cada uno de los seis grados de libertad cartesianos (X, Y, Z, Roll, Pitch, Yaw) se dispone de un par de botones + y -. Al presionar cualquiera de ellos, la función `send_delta(axis, sign)` construye un mensaje `TeachingDelta` con el incremento correspondiente al eje seleccionado y lo publica en `/dr/teaching_delta`. El nodo de *teaching* procesa el delta y publica la nueva pose objetivo en `/dr/tool_state`, que a su vez el nodo de cinemática inversa resuelve y envía al estado del robot.
- **Paso de jog (*step*).** Un *spinbox* configurable entre 0,001 y 0,1 m/rad determina el módulo del incremento aplicado en cada pulsación. Este valor se utiliza directamente en `send_delta`.
- **Modo de interpolación y velocidad.** Un combo de modo (*Articular* / *Cartesiano*) y un combo de velocidad (*Lento* / *Medio* / *Rápido*) acompañan cada waypoint capturado, de modo que la trayectoria resultante puede combinar segmentos con diferentes parámetros.
- **Captura de posición.** El botón “Capturar posición” lee la pose actual de la tabla de estado y la almacena en la lista interna de waypoints junto con el modo y la velocidad seleccionados en ese instante.
- **Gestión de waypoints.** El botón “Ver lista de waypoints” abre un diálogo (`WaypointDialog`) que muestra todos los puntos capturados. Desde este diálogo el usuario puede eliminar puntos individuales antes de confirmar la trayectoria.
- **Finalización.** El campo de texto de nombre y el botón “Finalizar” desencadenan el proceso de generación completo: se itera sobre la lista de waypoints llamando al servicio `/dr/gen_traj` para cada par de puntos consecutivos, las trayectorias parciales se concatenan y el resultado se persiste en disco llamando al servicio `/dr/save_traj`.

9.8. Interfaz gráfica

La interfaz gráfica fue desarrollada con *PyQt5* e integra en un único proceso tanto la lógica de comunicación con ROS 2 como la presentación visual. Para mantener la responsividad de la interfaz, el *spin* de ROS 2 se ejecuta en un hilo demonio (`threading.Thread`), mientras que todas las actualizaciones de widgets se realizan mediante señales Qt (`pyqtSignal`), evitando accesos al hilo principal desde el hilo de ROS 2. Las llamadas a servicios, que son bloqueantes, se realizan también en hilos separados sincronizados mediante `threading.Event`.

La ventana principal (`MainWindow`) se organiza en dos pestañas: *Teaching* (descrita en la sección 9.7) y *Ejecutar*.

Pestaña de ejecución

La pestaña *Ejecutar* permite cargar, recorrer y analizar trayectorias almacenadas. Sus componentes principales son:

- **Tabla de estado actual.** Idéntica a la de la pestaña *Teaching*: muestra $\mathbf{q}$ y la pose cartesiana en tiempo real.
- **Tabla de trayectorias disponibles.** Cada fila corresponde a una trayectoria almacenada e incluye columnas de nombre, fecha de creación, número de puntos y dos acciones:
 - *Recorrer*: carga la trayectoria llamando a `/dr/load_traj` y envía una solicitud a la acción `/dr/execute`. Durante la ejecución, la pestaña *Teaching* se deshabilita para evitar conflictos.
 - *Eliminar*: solicita confirmación al usuario y, de confirmarse, llama a `/dr/delete_traj` y refresca la tabla.
 - *Analizar*: habilitado únicamente tras ejecutar la trayectoria correspondiente. Abre el diálogo de análisis descrito a continuación.

- **Barra de progreso y etiqueta de estado.** Se actualizan en tiempo real a partir de los mensajes de *feedback* de la acción `/dr/execute`, que reportan el índice del punto en curso. Al completarse la acción, la etiqueta refleja el resultado (*éxito* o *error*).

A continuación, se presentan las imágenes correspondientes a cada pestaña.

DR - Panel de Control

Teaching Execute

	q1 / X	q2 / Y	q3 / Z	q4 / Roll	q5 / Pitch	q6 / Yaw
q (rad)	0.5915	0.1403	1.8593	2.7266	1.5736	-0.9894
pose	-0.0316	0.3335	0.2050	-3.1316	-0.0101	-3.1315

X

Y

Z

Roll

Pitch

Yaw

Ver lista de waypoints

Capturar posición

Nombre de trayectoria: ej: trayectoria_1

Finalizar

Figura 35: Interfaz gráfica de usuario en ROS 2 - Pestaña de teaching.

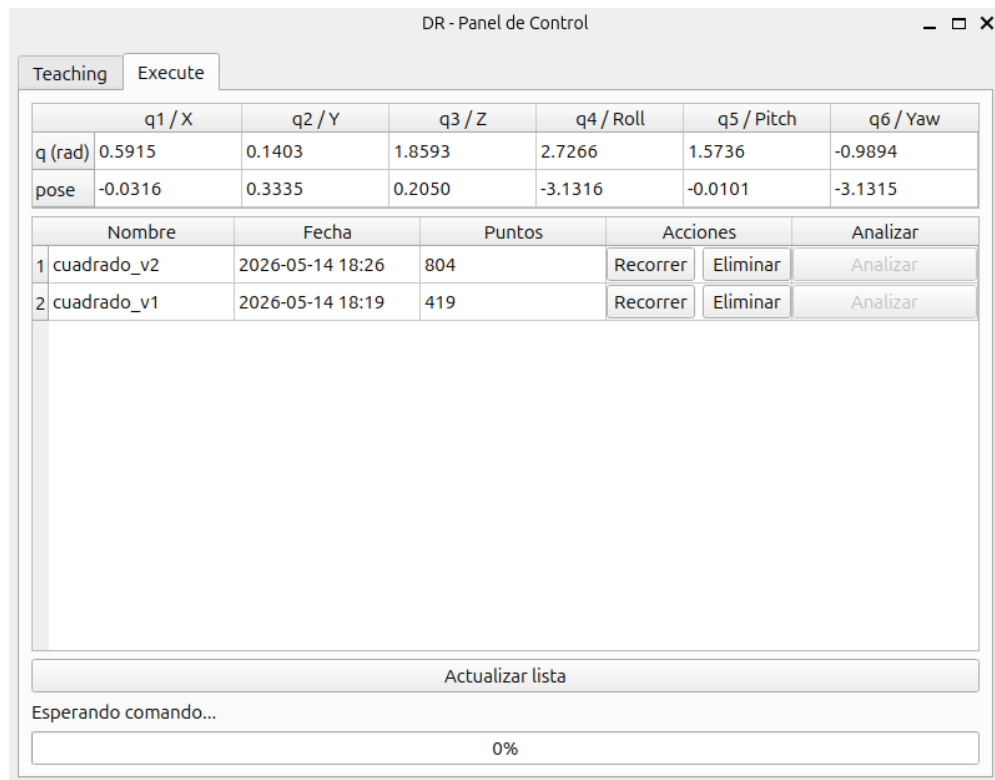


Figura 36: Interfaz gráfica de usuario en ROS 2 - Pestaña de ejecución.

Diálogo de análisis

Al finalizar la ejecución de una trayectoria, el botón *Analizar* abre el *AnalysisDialog*, un diálogo con cuatro pestañas de visualización:

1. **Coordenadas articulares:** evolución temporal de $q_1, \dots, q_6$ junto con los límites articulares $\pm q_{\text{máx}}$ y el perfil de velocidades y aceleraciones articulares con sus respectivos límites.
2. **Posición cartesiana del EF:** evolución de x, y, z y los ángulos de Euler ZYX (α, β, γ) computados mediante cinemática directa.
3. **Velocidad y aceleración lineal:** norma de la velocidad y aceleración lineales del efector final, calculadas a través del Jacobiano geométrico, con comparativa frente a los límites $v_{\text{máx}}$ y $a_{\text{máx}}$.
4. **Velocidad y aceleración angular:** norma de la velocidad y aceleración angulares del efector final, con comparativa frente a $\omega_{\text{máx}}$ y $\alpha_{\text{máx}}$.

El cálculo de cinemática directa para todas las muestras de la trayectoria se realiza en un hilo de fondo antes de renderizar los gráficos, de modo que la interfaz no se bloquea durante el proceso. Los gráficos se incrustan directamente en las pestañas del diálogo mediante *FigureCanvas* de *Matplotlib*, sin abrir ventanas externas.

Las siguientes imágenes muestran las 4 pestañas de análisis:

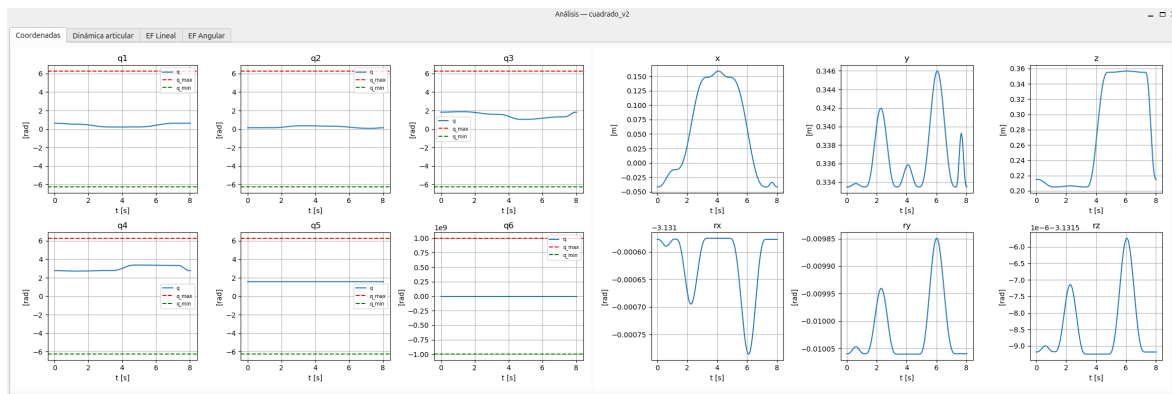


Figura 37: Interfaz gráfica de usuario - Pestaña de análisis: evolución de las coordenadas cartesianas del efector final y de las coordenadas articulares.

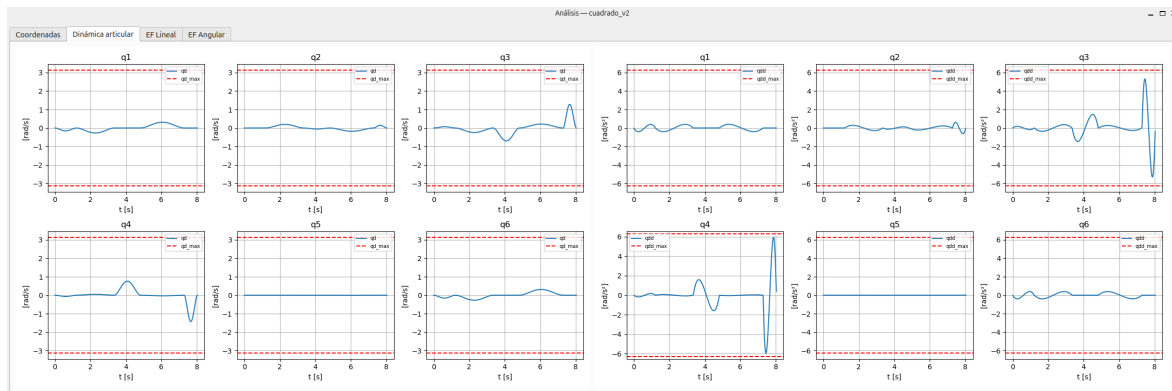


Figura 38: Interfaz gráfica de usuario - Pestaña de análisis: evolución de la velocidad articular y de la aceleración articular.

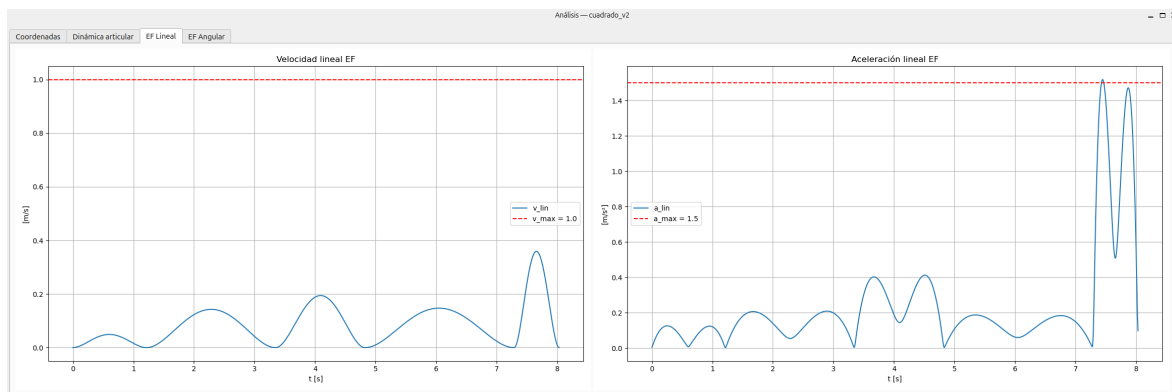


Figura 39: Interfaz gráfica de usuario - Pestaña de análisis: evolución de la velocidad lineal y angular del efector final.

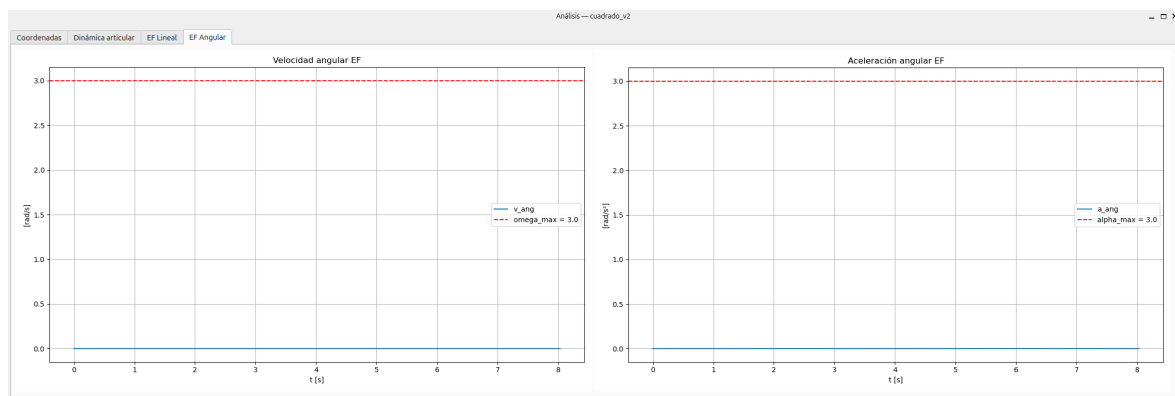


Figura 40: Interfaz gráfica de usuario - Pestaña de análisis: evolución de la aceleración lineal y angular del efector final.

Al igual que en el análisis realizado en MATLAB, se presentan los valores obtenidos junto con sus máximos definidos para verificar el funcionamiento.

10. Verificación de productividad en entorno ROS 2

Con el objetivo de evaluar el desempeño del sistema desarrollado, se busca verificar que el robot sea capaz de cumplir con el objetivo de productividad establecido: 300 transferencias por hora, lo que equivale a un período de ciclo de 12 segundos por transferencia. Para ello, se utiliza el sistema de *teaching* implementado en ROS 2 para describir la trayectoria completa de la tarea, ejecutarla en el entorno de simulación y analizar las variables cinemáticas resultantes.

10.1. Etapa 1: *Teaching* de la trayectoria

Utilizando la interfaz de *teaching* implementada con PyQt5, se capturan los nueve puntos vía que describen un ciclo completo de la tarea del robot. La figura 41 muestra las nueve posturas del manipulador correspondientes a cada punto capturado.

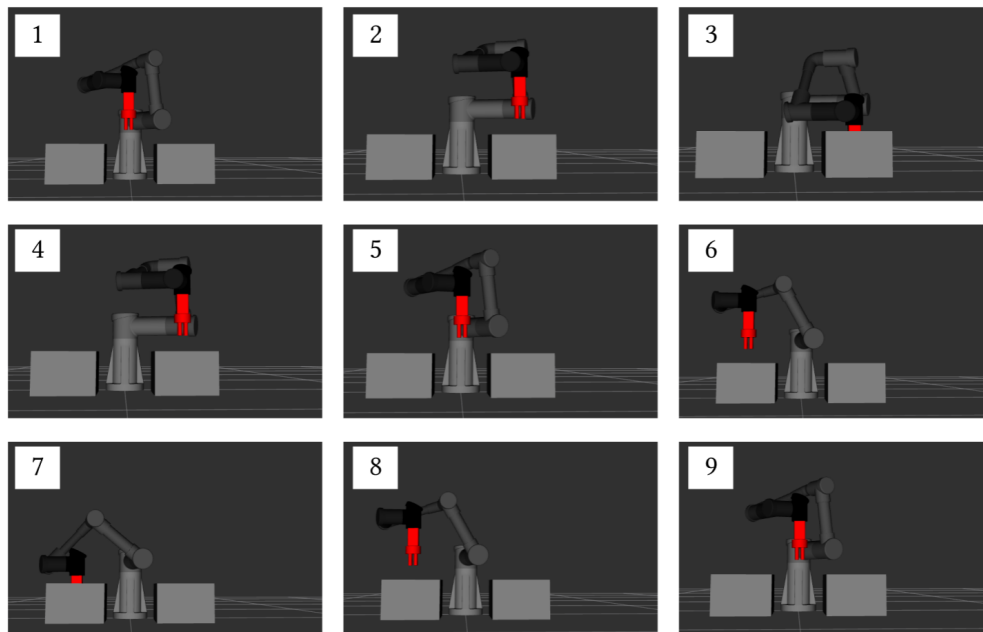


Figura 41: Nueve posturas del robot correspondientes a un ciclo completo de la tarea.

Los puntos vía capturados describen el ciclo de la siguiente manera:

1. **Postura de punto medio** — configuración inicial del ciclo, utilizada como postura de espera segura y alejada de los límites articulares.
2. **Postura sobre caja de *picking*** — el efector final se posiciona por encima de la caja de origen, previo al descenso.
3. **Postura dentro de caja de *picking*** — el efector desciende al interior de la caja de origen para realizar la toma del objeto.
4. **Postura sobre caja de *picking*** — el efector retorna a la posición elevada sobre la caja de origen con el objeto tomado.
5. **Postura de punto medio (intermedia)** — el robot transita por el punto medio para garantizar una trayectoria libre de interferencias entre origen y destino.
6. **Postura sobre caja de *placement*** — el efector se posiciona por encima de la caja de destino, previo al descenso.
7. **Postura dentro de caja de *placement*** — el efector desciende al interior de la caja de destino para depositar el objeto.

8. **Postura sobre caja de *placement*** — el efector retorna a la posición elevada sobre la caja de destino.
9. **Postura de punto medio (final)** — el robot regresa a la postura de espera, completando el ciclo.

La figura 42 muestra la lista de puntos vía cartesianos registrados en la interfaz, con su posición, modo de interpolación y perfil de velocidad de llegada.

	Posición	Modo	Speed	
1	-0.002, 0.333, 0.205, -3.132, -0.010, -3.132	Joint	Rápido	Eliminar
2	-0.202, 0.333, 0.205, -3.132, -0.010, -3.132	Joint	Rápido	Eliminar
3	-0.202, 0.333, 0.025, -3.132, -0.010, -3.132	Linear	Medio	Eliminar
4	-0.202, 0.333, 0.205, -3.132, -0.010, -3.132	Linear	Medio	Eliminar
5	-0.002, 0.333, 0.205, -3.132, -0.010, -3.132	Joint	Rápido	Eliminar
6	0.208, 0.333, 0.205, -3.132, -0.010, -3.132	Joint	Rápido	Eliminar
7	0.208, 0.333, 0.035, -3.132, -0.010, -3.132	Linear	Medio	Eliminar
8	0.208, 0.333, 0.205, -3.132, -0.010, -3.132	Linear	Medio	Eliminar
9	0.008, 0.333, 0.205, -3.132, -0.010, -3.132	Joint	Rápido	Eliminar

Figura 42: Lista de puntos vía cartesianos capturados mediante *teaching*, con modo de interpolación y velocidad de llegada.

10.2. Etapa 2: Recorrido de la trayectoria

Desde la pestaña *Ejecutar* de la interfaz gráfica implementada en PyQt5, se selecciona la trayectoria generada en la etapa anterior para cargar sus posiciones y ejecutar el recorrido. De forma simultánea, se valida visualmente la correcta definición de la trayectoria en el entorno de simulación RViz 2, verificando que el manipulador recorra las posturas definidas en el orden y con las características de interpolación establecidas durante el *teaching*.

10.3. Etapa 3: Análisis cinemático

Una vez finalizada la ejecución, se accede al análisis cinemático completo desde la opción *Analizar* disponible en la pestaña *Ejecutar* de la interfaz gráfica. El diálogo de análisis presenta cuatro pestañas con la evolución temporal de las variables cinemáticas del manipulador a lo largo del ciclo ejecutado.

La figura 43 muestra la evolución de las coordenadas articulares q_1 a q_6 y de la posición cartesiana del efector final $(x, y, z, \alpha, \beta, \gamma)$, donde α, β, γ son los ángulos de Euler ZYX.

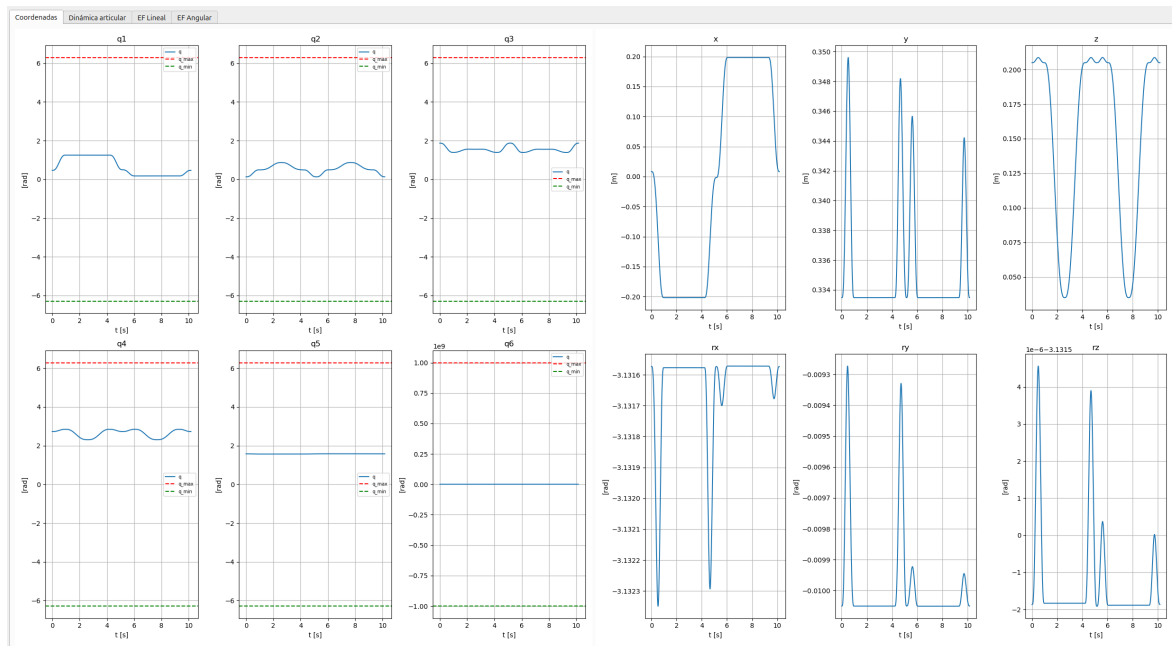


Figura 43: Evolución de posiciones articulares y coordenadas cartesianas del efector final durante el ciclo.

La figura 44 presenta las velocidades y aceleraciones articulares junto con sus límites definidos.

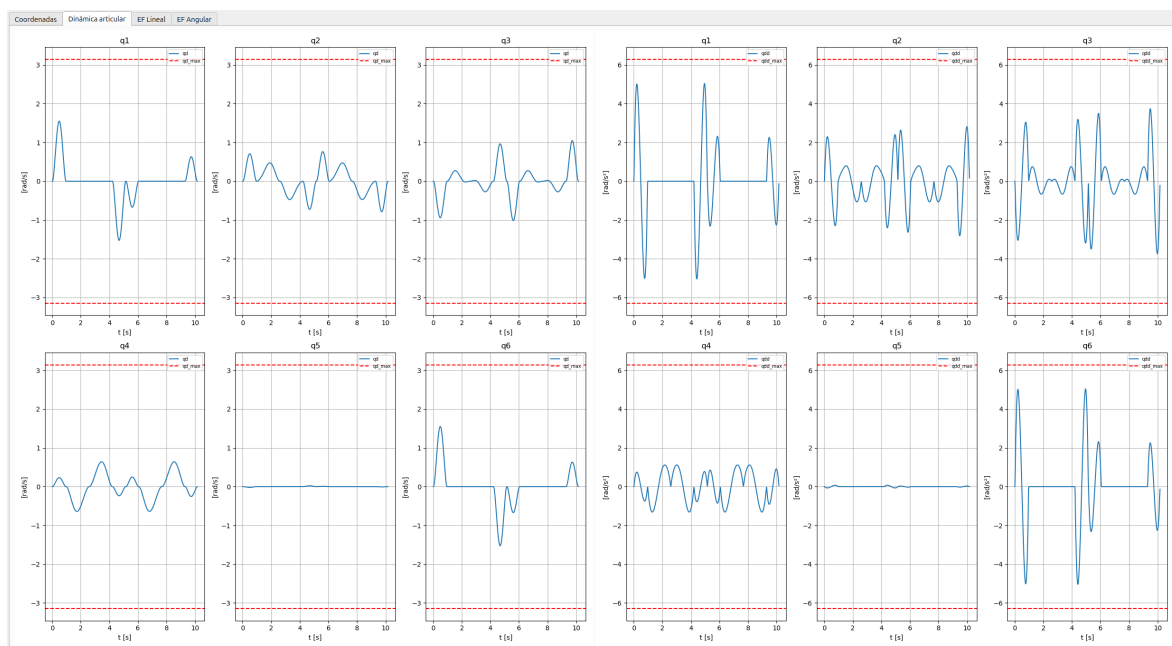


Figura 44: Evolución de velocidades y aceleraciones articulares durante el ciclo.

La figura 45 muestra la velocidad y aceleración lineales del efector final calculadas a través del Jacobiano geométrico, en comparación con los límites establecidos.

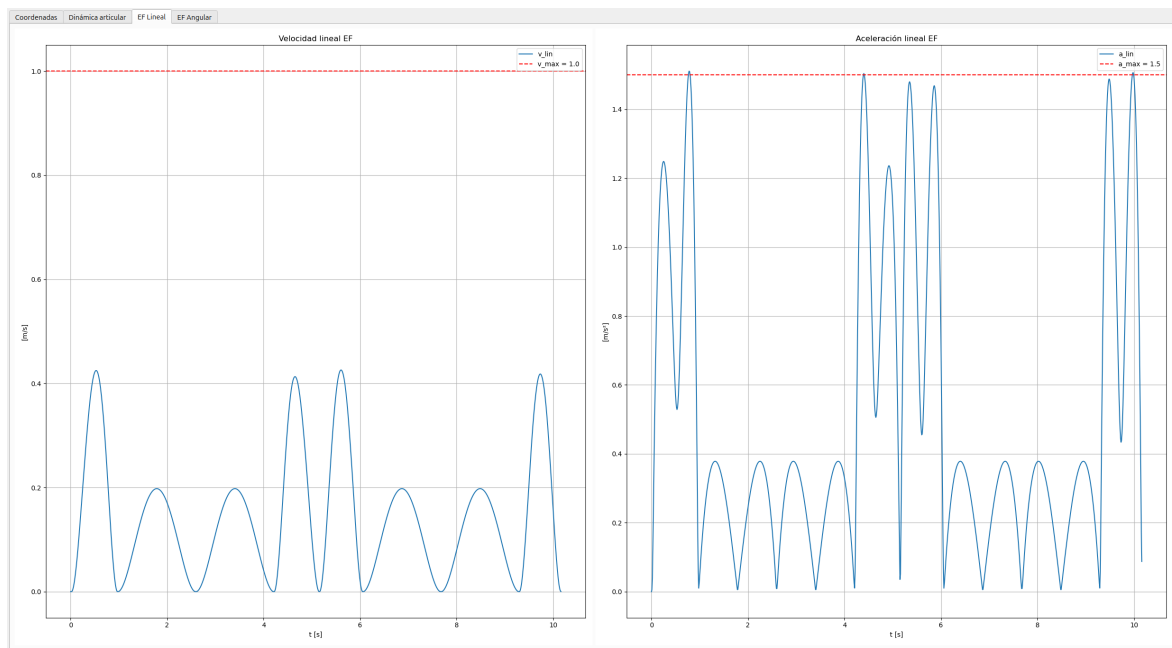


Figura 45: Evolución de velocidad y aceleración lineales del efector final durante el ciclo.

La figura 46 presenta la velocidad y aceleración angulares del efector final con sus respectivos límites.

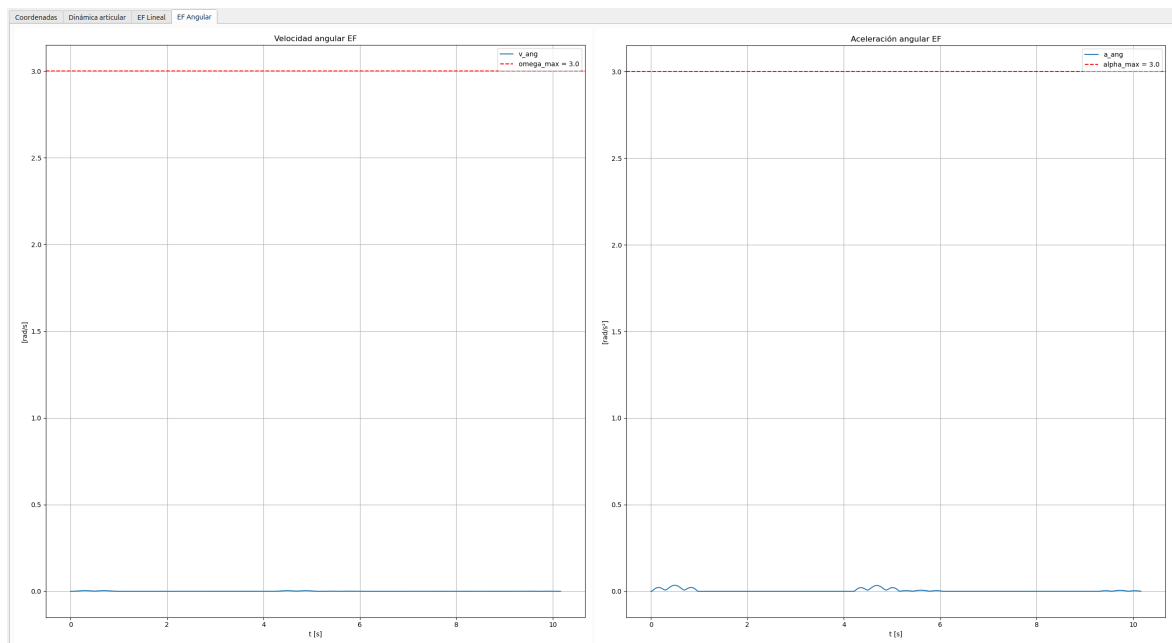


Figura 46: Evolución de velocidad y aceleración angulares del efector final durante el ciclo.

Del análisis se concluye que el ciclo operativo completo posee una duración de 10 segundos. Este tiempo corresponde al tiempo de recorrido de la trayectoria generada por el sistema de *teaching*, calculado a partir de la cantidad de muestras de la trayectoria a la frecuencia de control definida de 100 Hz, y no a una medición directa sobre hardware real. A partir de este valor, la productividad resultante es de $3600 \text{ s}/10 \text{ s} = 360$ transferencias por hora, lo que supera el objetivo de productividad establecido de 300 transferencias por hora. Asimismo, los perfiles de velocidad y aceleración tanto articulares como cartesianos se mantienen dentro de los límites cinemáticos definidos sin superarlos de forma considerable a lo largo de todo el ciclo.

11. Conclusiones y trabajo futuro

El presente trabajo logró cumplir con los objetivos planteados, abordando el modelado cinemático, la simulación y la integración en software de robótica de un manipulador serial de seis grados de libertad orientado a tareas de automatización logística.

En la etapa de modelado, se aplicó la convención Denavit-Hartenberg para formular la cinemática directa del manipulador y se obtuvo una solución analítica cerrada de la cinemática inversa. Este enfoque, alternativo a los métodos numéricos iterativos, resultó eficaz para el robot estudiado y permitió obtener hasta 512 configuraciones articulares válidas para una misma pose del efector final, brindando una amplia base para la selección de la solución más cercana a una configuración de referencia en cada instante de la trayectoria. Se identificaron y caracterizaron analíticamente las tres singularidades geométricas del manipulador: de hombro, de codo y de muñeca.

El módulo de generación de trayectorias incorporó interpolación en espacio articular y cartesiano, con un mecanismo de escalado automático de velocidades y aceleraciones basado en el Jacobiano geométrico, que permitió respetar los límites cinemáticos del sistema en las trayectorias evaluadas. La simulación de la tarea completa de *bin-to-bin transfer* en MATLAB permitió verificar el funcionamiento del conjunto, combinando segmentos de interpolación articular para los desplazamientos rápidos y segmentos de interpolación cartesiana para las maniobras de *picking* y *placing*.

El diseño CAD de los eslabones en *Fusion 360* y su integración como mallas STL al entorno de simulación permitieron verificar visualmente la coherencia geométrica entre el modelo cinemático y la representación tridimensional del robot.

Finalmente, la integración en ROS 2 trasladó la cadena cinemática a una arquitectura de nodos distribuida. En ROS 2 se implementaron módulos propios de cinemática y generación de trayectorias e interfaces de comunicación, mientras que MATLAB se utilizó como entorno de modelado y validación apoyado en la *Robotics Toolbox* de Peter Corke, no disponible en el entorno Python de ROS 2. Se completó además un modelo URDF derivado directamente de la parametrización DH, una visualización del robot en RViz 2 y una interfaz gráfica de usuario en PyQt5 que centraliza el movimiento manual, la enseñanza de puntos, la gestión de trayectorias y el análisis de variables cinemáticas.

Se verificó el funcionamiento del sistema en este entorno y se evaluó el cumplimiento de los objetivos de productividad establecidos para la tarea. Como limitación identificada en esta etapa, el servicio de generación de trayectorias acepta un único punto por solicitud, lo que impide la interpolación continua entre puntos vía consecutivos y obliga al robot a detenerse en cada punto intermedio, a diferencia del comportamiento logrado en MATLAB.

Trabajo futuro

A partir del estado actual del desarrollo, se identifican las siguientes líneas de extensión prioritarias:

1. **Interpolación continua entre puntos vía en ROS 2.** Rediseñar el servicio `/dr/gen_traj` para que acepte una lista completa de puntos vía en una única solicitud, replicando el comportamiento de `mstraj` en MATLAB. Esto eliminaría la detención del robot en cada punto intermedio y permitiría trayectorias continuas y fluidas en el entorno ROS 2.
2. **Replanificación de trayectorias en tiempo real.** Incorporar un módulo de replanificación que, ante cambios en el entorno o desviaciones respecto a la trayectoria nominal, genere una trayectoria alternativa sin detener la ejecución. Una primera aproximación realista consiste en monitorear el determinante del Jacobiano durante la ejecución y disparar una replanificación automática al detectar proximidad a configuraciones singulares.
3. **Detección y evasión de colisiones.** Integrar un módulo de verificación de colisiones que opere tanto sobre objetos del entorno como sobre la propia geometría del robot (*self-collision*). Una implementación realista puede basarse en la representación simplificada del robot mediante esferas o cápsulas envolventes y en el uso de la biblioteca *FCL* (*Flexible Collision Library*), disponible en el ecosistema ROS 2.
4. **Mejoras de simulación en ROS 2.** Extender la simulación para incorporar el comportamiento completo de la tarea: activación y desactivación del *gripper* durante las maniobras de *picking* y *placing*, gestión del estado de la herramienta y visualización en RViz 2 del objeto manipulado a

lo largo de la trayectoria. Esto permitiría una validación más fiel del ciclo completo de trabajo del manipulador.

5. **Robustez del sistema.** Mejorar el manejo de errores en los nodos de ROS 2 incorporando reintentos con *timeout*, recuperación ante fallos de servicio y retroalimentación más detallada hacia la interfaz gráfica. Incorporar métricas de tiempo de ejecución por segmento de trayectoria para facilitar el ajuste fino de los perfiles de velocidad en función de los requerimientos de productividad.
6. **Visión artificial y generación de trayectorias reactivas.** Integrar un módulo de visión artificial —basado en una cámara RGB-D y un algoritmo de estimación de pose— que permita detectar en tiempo de ejecución la posición y orientación real de cada pieza dentro de la caja de origen, así como el espacio disponible en la caja de destino. A partir de esta información, el generador de trayectorias produciría una *trayectoria reactiva* adaptada a la pose detectada en cada ciclo, en lugar de depender de posiciones predefinidas. Esta extensión completaría el sistema de transferencia descrito en la sección 3, donde la visión artificial y la trayectoria reactiva quedaron reservadas para desarrollos posteriores.
7. **Planificador de trayectoria nominal/reactiva.** Desarrollar un planificador de alto nivel que gestione la transición entre la *trayectoria nominal* —precomputada y optimizada para maximizar la productividad— y la *trayectoria reactiva* —generada en línea a partir de la detección visual—. El planificador determinaría cuándo activar cada modo en función del estado del sistema y garantizaría continuidad en velocidad y aceleración durante la conmutación entre ambos regímenes. Esta capa de coordinación constituye el tercer componente del sistema de transferencia identificado en el alcance original del trabajo.

12. Uso de agentes de inteligencia artificial

A lo largo del desarrollo de este trabajo se ha utilizado inteligencia artificial generativa como herramienta de apoyo en múltiples etapas del proceso. Su incorporación permitió automatizar y mejorar tareas de diversa índole: el desarrollo y refinamiento de algoritmos, la generación de visualizaciones, la verificación de errores lógicos en el código, la redacción y estructuración del presente informe, entre otras. En todos los casos, el uso de estas herramientas se realizó de forma supervisada, validando los resultados obtenidos antes de su incorporación al trabajo.

Las herramientas utilizadas fueron:

- Claude (Anthropic)
- Gemini (Google)

12.1. Metodología de uso de inteligencia artificial

El uso de inteligencia artificial a lo largo del proyecto se ha llevado a cabo mediante un enfoque de *cápsulas*: las tareas encomendadas a los agentes fueron deliberadamente limitadas y acotadas tanto en objetivo como en alcance. Esta metodología busca evitar que la incorporación de IA conduzca a la generación de código complejo de desarrollo ajeno, que dificulte la comprensión del programa, complique futuras modificaciones y entorpezca la búsqueda de errores. Bajo este enfoque, cada interacción con el agente se planteó como una tarea concreta y bien definida, cuyo resultado fue revisado e integrado de forma consciente por el autor.

12.2. MATLAB

En el desarrollo de los módulos de MATLAB, el uso de agentes de IA se limitó a la búsqueda de errores lógicos en el código, la mejora de las visualizaciones obtenidas y la discusión sobre posibles mejoras, cambios e ideas para expandir el trabajo o refinar los resultados. Los algoritmos, la lógica de cada módulo y las decisiones de diseño fueron desarrollados de forma íntegra por el autor, utilizando al agente como interlocutor crítico más que como generador de soluciones.

12.3. ROS 2

En el desarrollo del sistema en ROS 2, el uso de agentes de IA fue similar al descripto para MATLAB, con el agregado de que se recurrió a ellos para la identificación y resolución de errores de mayor complejidad técnica, tales como problemas de sincronidad y paralelismo entre hilos de ejecución. En estos casos, las soluciones propuestas por el agente fueron analizadas en detalle por el autor con el objetivo de comprender el origen del problema y adquirir el conocimiento asociado, evitando incorporar correcciones sin entender su fundamento.

Como excepción al enfoque de cápsulas descripto, la implementación de la interfaz gráfica (GUI) constituye un desarrollo realizado íntegramente por un agente de IA. Dicho desarrollo fue analizado por el autor a grandes rasgos para comprender su funcionamiento general, pero se considera un material cuya implementación interna queda fuera del alcance técnico detallado del presente trabajo.

12.4. Desarrollo del informe

Para la redacción del presente informe se utilizaron agentes de IA con el objetivo de automatizar la generación de texto a partir de explicaciones provistas por el autor. El enfoque consistió en transmitir al agente la idea, concepto o descripción a desarrollar, para que este produzca un texto coherente, bien estructurado y sin errores gramaticales, incluyendo la referenciación de imágenes, secciones y ecuaciones dentro del documento. Posteriormente, el autor realizó una verificación de la calidad técnica del texto generado, corrigiendo o ajustando aquellos aspectos que no reflejaran con precisión el contenido del trabajo.

12.5. Uso general

De forma transversal a todas las etapas del proyecto, los agentes de IA fueron utilizados como herramienta de consulta general: para obtener explicaciones sobre conceptos técnicos, recibir sugerencias frente a decisiones de diseño, explorar alternativas de implementación y ampliar el entendimiento de temas relacionados con el trabajo.

12.6. Formato de implementación

Las herramientas de IA no se utilizaron de forma uniforme en todos los contextos. En los apartados de MATLAB, ROS 2 y desarrollo del informe se empleó **Claude Code** (interfaz de línea de comandos de Anthropic, integrada al entorno de desarrollo) en conjunto con **Claude** vía web (claude.ai). Para el uso general — consultas, explicaciones y sugerencias — se recurrió a **Claude** vía web junto con **Google Gemini** (gemini.google.com).

A. Verificación y Validación de los Módulos de Software de MATLAB y ROS 2

A.1. Cinemática Inversa

La verificación del correcto funcionamiento del módulo de cinemática inversa se realiza mediante el script `test_inv_kinematics.m`. Este incluye 3 funcionalidades:

1. **Verificación redundante:** Se define una coordenada articular, se calcula la cinemática directa para obtener la coordenada cartesiana del efector final, sobre esta última se calcula la cinemática inversa con el módulo desarrollado. Se presentan los resultados dejando en evidencia la coordenada articular inicial y el conjunto obtenido mediante cinemática inversa, también se calcula la cinemática directa para cada solución obtenida permitiendo otro contraste de resultados. Finalmente, se realiza un resumen estadístico del error entre la coordenada cartesiana de entrada y las coordenadas cartesianas obtenidas a partir de cada solución articular, para verificar que el error se mantenga dentro de un rango aceptable.

```
% ==INPUT ==
% Coordenadas articulares ingresadas:
q = [1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078];
% Las coordenadas cartesianas de la consigna (calculadas mediante DK) son
:
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
% ==OUTPUT==
% Las coordenadas articulares de la solución obtenidas de la cinemática
    inversa son (solo algunas):
1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078
-4.4474    0.4922   -1.9151   -0.5557    1.7815    1.2078
-4.4474   -0.2860   -1.5354    2.9844    4.5017    4.3494
3.2637    0.2942   -4.8523   -3.1284    1.9973    2.7157
% Las coordenadas cartesianas de la solución obtenidas son las siguientes
    (se presentan solo algunas, todas coinciden):
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087

% --- Resumen estadístico IK (poses aleatorias) ---
Poses testeadas:    300
Sin solución IK:    0

Poses verificadas:  300 (100.0%)
Error posición      - max: 3.66e-06 m    prom: 2.06e-06 m
Error orientación   - max: 1.10e-05 rad   prom: 6.20e-06 rad
```

2. **Cálculo directo:** Esta funcionalidad no se define específicamente para realizar una verificación sino para conocer las coordenadas articulares de la solución de la cinemática inversa a partir de la coordenada cartesiana ingresada de forma directa. Los resultados se presentan de la misma forma que el ítem 1 dado que solo se realiza un *override* durante ejecución de la coordenada cartesiana de entrada.
3. **Verificación cíclica:** Esta funcionalidad permite, hasta que se detenga la ejecución del programa, verificar cíclicamente puntos aleatorios mediante un esquema de verificación similar al del ítem 1. De forma adicional, se calcula la norma entre coordenadas cartesianas de salida y de llegada, ante una diferencia mayor a un umbral determinado en cualquiera de las soluciones

pertenecientes al conjunto de solución obtenido, se detiene el programa para verificar el error y poder analizarlo.

```
% ==INPUT ==
% LOOP -> Probando posición articular
-3.2460    -1.2075    -5.0711    -4.6248     5.5550    -6.2832

% LOOP -> Las coordenadas cartesianas consigna son:
x = 0.10246 | y = -0.40972 | z = 0.45372 | alpha = 3.0035 | beta =
    0.83783 | gamma = 1.3635
% ==OUTPUT==
% Las coordenadas articulares solución obtenidas de la cinemática inversa
son (solo algunas):
3.0372     0.7669    -1.7414    -0.5041     0.7282    -3.1416
-3.2460    -5.5163     4.5418     5.7791    -5.5550    -3.1416
-3.2460    -6.2786     5.0711    -3.4127    -0.7282     0
-2.2848     0.0199     0.9305    -3.2374     1.0729    -1.9493
% Resultado de norma:
OK
OK
OK
OK
```

NOTA: En los resultados presentados, se mencionan solo una pequeña porción de toda la información devuelta por el script.

A.2. Singularidades

Para verificar que los patrones que conducen a singularidades determinados visualmente son correctos, se elabora el script `test_singularities.m`. En él se encuentra un ciclo *while* infinito, solo terminable mediante finalización de ejecución.

Para la singularidad de codo y de muñeca, se definen valores aleatorios para todas las variables articulares que no estén involucradas en el patrón de la singularidad, para las que sí están involucradas, se define un valor aleatorio dentro del conjunto de aquellos valores que puedan provocar la singularidad según las condiciones definidas en 5.5.

Para la singularidad de hombro, debido a la presencia de 3 incógnitas en una misma ecuación, simplemente se busca verificar la observación, para esto, se dan valores aleatorios dentro de un rango de $\pm 10^\circ$ y con signo opuesto para q_2 y q_3 , finalmente se busca el valor de q_4 que cumpla con la condición de proyección nula. El motivo por el que los valores para q_2 y q_3 están dentro del rango $\pm 10^\circ$ y de signo opuesto, es para evitar que la proyección no sea anulable geoméricamente mediante q_4 .

Para cada caso, se calcula el jacobiano en la configuración determinada y su determinante, solo en caso de que el determinante del jacobiano sea mayor al valor de umbral, de modo que no se considere cercano a una singularidad, se detiene la ejecución y se muestra información, caso contrario, la ejecución se repite cíclicamente informando cuando se completa cada ciclo. Un ciclo equivale al análisis de una configuración para el codo, una para la muñeca y una para el hombro.

A continuación se detalla un ejemplo para cada salida.

```
% == OUTPUT DE SINGULARIDAD VERIFICADA (TODAS CONFIGURACIONES SINGULARES)
==
Ciclo completo - todas las posiciones fueron singulares - continuando...
% == OUTPUT DE NO SINGULARIDAD ==
Posición articular aleatoria - no singular - hombro
1.7748    -0.1075     0.0635     0.4317    -0.3929     3.0684
Determinante del Jacobiano: 7.897e-20
-> Detención por posición no singular - enter para continuar
```

El segundo bloque muestra la salida de una detención por configuración no singular. Aunque el valor del determinante ($7,897 \times 10^{-20}$) es cercano a cero en términos absolutos, este ejemplo se obtuvo reduciendo excesivamente el umbral de comparación a un valor extremadamente pequeño, lo que provocó que la configuración fuera clasificada como no singular y la ejecución se detuviera. En condiciones

normales de operación, el umbral está calibrado para que configuraciones con determinantes de ese orden queden correctamente clasificadas como singulares y el ciclo continúe sin interrupciones.

A.3. Generación de trayectorias

En la generación de trayectorias se busca verificar dos aspectos fundamentales: que la trayectoria generada efectivamente interpole entre los puntos vía definidos y que el perfil de velocidades y aceleraciones resultante respete los límites cinemáticos establecidos. Para esto se elabora el script `test_gen_traj.m` que contiene el siguiente procedimiento de verificación: Se definen manualmente una serie de puntos vía con sus respectivos modos de interpolación (articular o cartesiano) y se genera la trayectoria utilizando el módulo desarrollado y se verifica:

1. **Interpolación entre puntos vía:** Se verifica que la trayectoria resultante efectivamente pase por cada uno de los puntos vía definidos, para esto, se calcula la distancia entre cada punto vía y el punto de la trayectoria correspondiente al mismo tiempo, verificando que esta distancia sea menor a un umbral determinado. En caso de detectar una cercanía lo suficientemente pequeña, se considera que el punto vía ha sido interpolado correctamente y se incrementa un contador. La verificación será positiva siempre que el contador sea mayor o igual al número total de puntos vía definidos.
2. **Cumplimiento de límites cinemáticos:** Se imprime la comparativa entre valores máximos, perfiles cinemáticos iniciales y perfiles cinemáticos escalados para cumplir con límites. Se verifica: velocidad y aceleración angular de articulaciones, velocidad y aceleración lineal y angular de efector final. De este modo, la verificación será visual.

Como complemento numérico a la verificación visual descripta, se presenta a continuación la salida del script `test_gen_traj.m` para una trayectoria de prueba, donde se verifica el alcance de los puntos vía y se contrastan los límites articulares y cinemáticos establecidos con los valores máximos efectivamente alcanzados a lo largo de la trayectoria escalada:

```
N_alcanzados =

60

Todos los puntos de control fueron alcanzados.
--- Verificación de límites articulares ---
q1: límite = [-6.283, 6.283] rad | alcanzado = [-1.298, 1.391] rad
q2: límite = [-6.283, 6.283] rad | alcanzado = [-0.403, 0.548] rad
q3: límite = [-6.283, 6.283] rad | alcanzado = [-1.914, 1.021] rad
q4: límite = [-6.283, 6.283] rad | alcanzado = [3.155, 3.508] rad
q5: límite = [-6.283, 6.283] rad | alcanzado = [-1.584, 1.563] rad
q6: límite = [-17453.293, 17453.293] rad | alcanzado = [-0.190, 0.785] rad

--- Verificación de velocidades y aceleraciones articulares ---
qd1: límite = 3.142 rad/s | alcanzado = 0.530 rad/s
qd2: límite = 3.142 rad/s | alcanzado = 0.151 rad/s
qd3: límite = 3.142 rad/s | alcanzado = 0.758 rad/s
qd4: límite = 6.283 rad/s | alcanzado = 0.089 rad/s
qd5: límite = 6.283 rad/s | alcanzado = 0.792 rad/s
qd6: límite = 6.283 rad/s | alcanzado = 0.261 rad/s
qdd1: límite = 13.963 rad/s^2 | alcanzado = 1.208 rad/s^2
qdd2: límite = 13.963 rad/s^2 | alcanzado = 0.343 rad/s^2
qdd3: límite = 13.963 rad/s^2 | alcanzado = 1.728 rad/s^2
qdd4: límite = 13.963 rad/s^2 | alcanzado = 0.203 rad/s^2
qdd5: límite = 13.963 rad/s^2 | alcanzado = 1.807 rad/s^2
qdd6: límite = 13.963 rad/s^2 | alcanzado = 0.596 rad/s^2

--- Verificación de velocidades y aceleraciones del efector final ---
|v|: límite = 1.000 m/s | alcanzado = 0.330 m/s
|a|: límite = 1.500 m/s^2 | alcanzado = 0.739 m/s^2
|w|: límite = 3.000 rad/s | alcanzado = 1.235 rad/s
|alpha|: límite = 3.000 rad/s^2 | alcanzado = 3.098 rad/s^2
```

Se observa que la totalidad de los valores articulares y cartesianos alcanzados se mantiene dentro de los límites establecidos, con la única excepción de la aceleración angular del efector final, cuyo valor máximo (3.098 rad/s^2) supera marginalmente el límite definido (3.000 rad/s^2) en un instante puntual de la trayectoria, sin representar una superación considerable. A continuación puede observarse el impacto de los efectos de escalado en el perfil de velocidad en ambos dominios, articular y cartesiano:

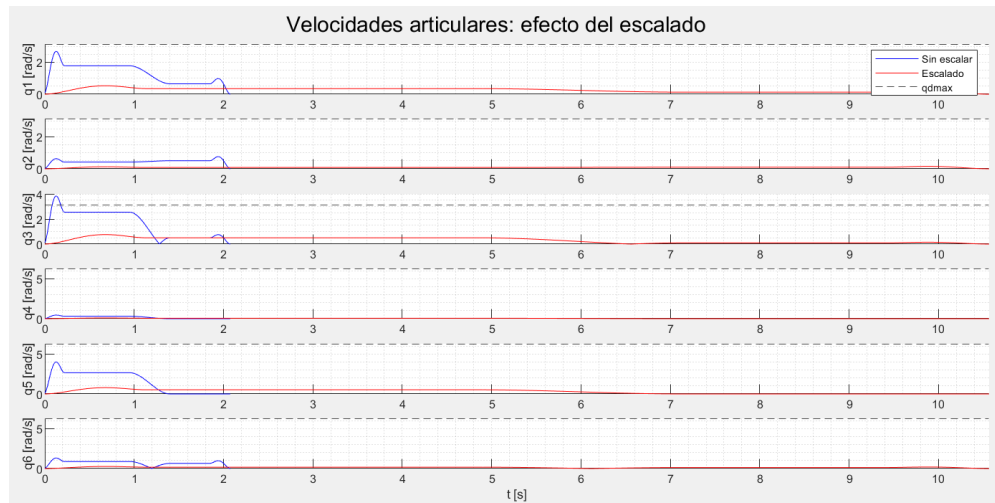


Figura 47: Comparativa entre perfiles de velocidad articular antes y después del escalado para cumplimiento de límites.

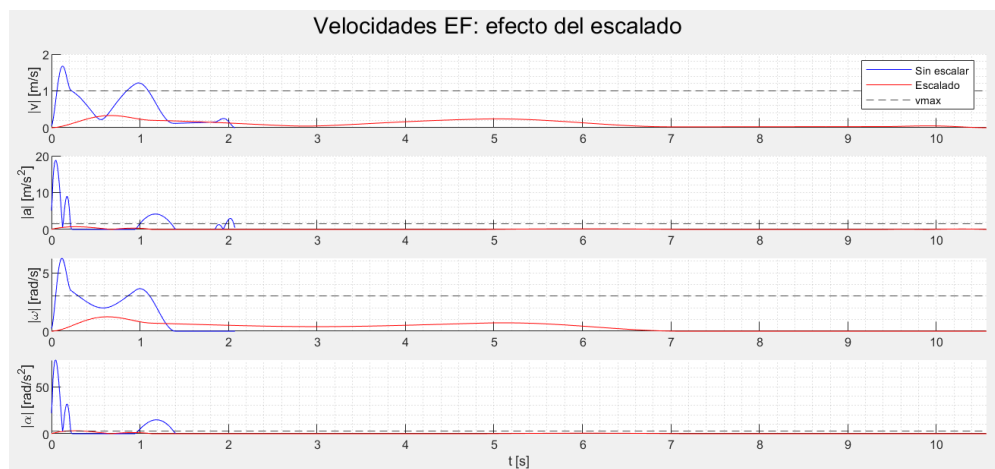


Figura 48: Comparativa entre perfiles de velocidad cartesiana del efector final antes y después del escalado para cumplimiento de límites.

A.4. Integración en ROS 2

La verificación del sistema en ROS 2 se realizó de forma progresiva a lo largo del desarrollo, probando cada nodo de forma aislada antes de integrarlo con el resto del sistema. La metodología consistió en llamar directamente a los servicios y tópicos desde la terminal, sin depender de la interfaz gráfica, lo que permitió validar cada componente de forma independiente y detectar errores de comunicación, serialización y lógica en etapas tempranas.

A continuación se detallan los comandos más representativos utilizados durante este proceso:

■ Inspección del grafo de nodos y tópicos activos:

```
ros2 node list
ros2 topic list
ros2 service list
```

- **Monitoreo del estado del robot en tiempo real** (pose articular y cartesiana publicada por el nodo de cinemática):

```
ros2 topic echo /dr/robot_state
```

- **Llamada al servicio de cinemática directa** con una configuración articular manual:

```
ros2 service call /dr/solve_fk dr_interfaces/srv/SolveFK \
  "{q:[0.0,0.5,-1.0,0.0,1.5,0.0]}"
```

- **Llamada al servicio de cinemática inversa** a partir de una pose cartesiana objetivo:

```
ros2 service call /dr/solve_ik dr_interfaces/srv/SolveIK \
  "{pose:[-0.22,0.3,0.2,0.01,3.1416,0.01], \
  q0:[0.0,0.0,0.0,0.0,1.5708,0.0],force:false}"
```

- **Generación de una trayectoria** desde la configuración actual hasta un punto cartesiano objetivo:

```
ros2 service call /dr/gen_traj dr_interfaces/srv/GenTraj \
  "{pose:[0.0,0.3,0.25,0.01,3.1416,0.01],mode:1,speed_profile:1}"
```

- **Guardado, listado y eliminación de trayectorias** en el nodo de almacenamiento:

```
ros2 service call /dr/save_traj dr_interfaces/srv/SaveTraj \
  "{name: 'tarea_1', points: 2, \
  q_traj: [0.0, 0.5, -1.2, 0.3, 1.1, 0.0, \
  0.1, 0.6, -1.1, 0.4, 1.0, 0.1]}"
ros2 service call /dr/list_trajs dr_interfaces/srv/ListTrajs "{}"
ros2 service call /dr/delete_traj dr_interfaces/srv/DeleteTraj "{name:'tarea_1'}"
```

- **Envío de un delta de teaching** para verificar el desplazamiento incremental del efector final:

```
ros2 topic pub --once /dr/teaching_delta \
  dr_interfaces/msg/TeachingDelta \
  "{dx:0.01,dy:0.0,dz:0.0,droll:0.0,dpitch:0.0,dyaw:0.0}"
```

- **Ejecución de una trayectoria** mediante la acción `/dr/execute`: el goal recibe la trayectoria aplanada `q_traj` y el entero `points`, por lo que desde consola se combina con la carga previa vía `/dr/load_traj`. El feedback reporta el índice del punto en curso y el progreso porcentual:

```
ros2 action send_goal /dr/execute \
  dr_interfaces/action/Execute \
  "{q_traj:[...],points:N}" --feedback
```

Este flujo de verificación incremental garantizó que cada nodo cumpliera su contrato de interfaz antes de ser integrado al sistema completo, reduciendo significativamente el tiempo de depuración en las etapas de integración.

Referencias

- [1] Grand View Research.
Global Warehouse Robotics Market Size & Outlook, 2023–2030.
2024.
Disponible en: [https://www.grandviewresearch.com/horizon/outlook/
warehouse-robotics-market-size/global](https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market-size/global)
- [2] Grand View Research. *Warehouse Robotics Market Outlook: United States*. 2024. Disponible en: [https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/
united-states](https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/united-states)
- [3] Grand View Research. *Warehouse Robotics Market Outlook: China*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/china>
- [4] Grand View Research. *Warehouse Robotics Market Outlook: Latin America*. 2024. Disponible en: [https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/
latin-america](https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/latin-america)
- [5] Grand View Research. *Warehouse Robotics Market Outlook: Brazil*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/brazil>